Bonus Çalışma Kitabı

Qt Widget Referansı – Cevap Anahtarı

Bölüm 8 ile Bölüm 9 Arası Bonus Materyal

C++20 + Qt 6.5+

Cevap Anahtarı

Bu doküman öğretim üyesi ve kendini test etmek isteyen öğrenciler içindir. Her widget'ın alıştırma cevapları ve kategori bitiş quiz'i cevapları bu kitapta yer alır.

İçindekiler

1 Temel Widget'lar

Bu kategoride Qt arayüzünün *en temel yapıtaşlarını* bulacaksın: her şeyin atası QWidget, metin göstermek için QLabel, kullanıcı etkileşiminin sembolü olan butonlar ve çerçeveleme için QFrame. Bölüm 4'te *(bkz. kitap)* ilk kez QPushButton ve QLabel'ı kullanmıştık; burada o bilgileri toparlıyor ve genişletiyoruz.

1.1 QWidget

QObject → QWidget

Qt arayüzündeki tüm görsel elemanların ortak atasıdır. Bir QWidget hem bağımsız bir pencere (*top-level*) hem de başka bir widget'ın çocuğu (*child*) olarak yaşayabilir. Üst sınıf olduğu için olay (event) sistemi, paint, focus, klavye/fare yönlendirmesi gibi tüm temel davranışlar burada tanımlıdır. OBYS örneğinde bir form sayfası, bir kart bileşeni veya basitçe boş bir arka plan paneli birer QWidget'tir.

Önemli Properties

Property	Tip	Açıklama
geometry	QRect	Konum + boyut (x, y, w, h)
minimumSize	QSize	İzin verilen en küçük boyut
maximumSize	QSize	İzin verilen en büyük boyut
visible	bool	Ekranda görünür mü
enabled	bool	Olay alabilir mi
styleSheet	QString	Qt Style Sheet (CSS benzeri)
windowTitle	QString	Pencere başlığı (top-level ise)
windowIcon	QIcon	Pencere ikonu (top-level ise)

Signals

- `customContextMenuRequested(const QPoint&)` – Sağ tık menüsü istendi
- `windowTitleChanged(const QString&)` – Pencere başlığı değişti
- `windowIconChanged(const QIcon&)` – Pencere ikonu değişti

Slots

- `show()` – Widget'ı görünür yap
- `hide()` – Widget'ı gizle
- `close()` – Widget'ı kapat (top-level ise pencereyi)
- `setEnabled(bool)` – Etkin/pasif yap
- `update()` – Yeniden çizim talep et (asenkron)
- `repaint()` – Hemen yeniden çiz (senkron)
- `setFocus()` – Klavye odağını al

C++ - Mini örnek: Bos pencere

```
auto* w = new QWidget;
w->setWindowTitle(tr("OBYS - Bos Sayfa"));
w->resize(400, 300);
w->show();
```

Alıştırma

1. `show()` ile `setVisible(true)` arasında pratik bir fark var mı? Hangisini ne zaman tercih edersin?

Cevabın: _____

2. Bir widget'ı ekrandan tamamen kaldırmadan "arka plana göndermek" (yani altındaki widget'ları görünür yapmak) için hangi slot/yöntem kullanılır?

Cevabın: _____

Cevap Anahtarı

1. Pratikte aynıdır: `show()` aslında `setVisible(true)`'ı çağırır. Okunabilirlik için `show()/hide()` tercih edilir; `setVisible(bool)` ise bir flag değişkenine bağlı dinamik gösterimde daha uygundur.
2. `lower()` slot'u widget'ı kardeş yığınında en alta indirir. `hide()` kullanmazsın çünkü yer kaplamaya devam etmesi istenir; sadece üstüne başka widget alınır. Karşıtı `raise()`'tir.

1.2 QLabel

QObject → QWidget → QFrame → QLabel

Metin, sayı veya resim göstermek için kullanılan en temel "çıktı" widget'ıdır. Sade düz metin yanında *Rich Text* (HTML alt kümesi) ve *Markdown* render edebilir; tıklanabilir köprüler içerebilir. OBYS'de öğrencinin harf notunu büyük puntoyla göstermek, ders adı yazmak ya da bir bilgilendirme metni sunmak için ideal seçimdir. QFrame'den türediği için çerçeve özellikleri (frameShape, frameShadow) doğrudan kullanılır.

Önemli Properties

Property	Tip	Açıklama
text	QString	Gösterilen metin
pixmap	QPixmap	Gösterilen resim
alignment	Qt::Alignment	Metin/resim hizalaması
wordWrap	bool	Otomatik satır kaydırma
textFormat	Qt::TextFormat	Plain / Rich / Markdown / Auto
openExternalLinks	bool	Linke tıkla = sistem tarayıcısı
indent	int	Metin girintisi (px)

Signals

- `linkActivated(const QString& link)` – Köprüye tıklandı
- `linkHovered(const QString& link)` – Köprü üzerine gelindi

Slots

- `setText(const QString&)` – Metni değiştir
- `setPixmap(const QPixmap&)` – Resmi ayarla
- `setNum(int)` / `setNum(double)` – Sayı göster
- `clear()` – İçeriği temizle

C++ - Mini örnek: OBYS harf notu rozeti

```
auto* notLabel = new QLabel(tr("AA"), this);
notLabel->setAlignment(Qt::AlignCenter);
notLabel->setStyleSheet(
    "font-size: 32px; font-weight: bold;"
    "color: white; background: #2ecc71;"
    "border-radius: 8px; padding: 12px;");
```

Alıştırma

1. Bir QLabel'in HTML etiketlerini (`<b>`, `<i>`) yorumlaması için `textFormat` hangi değere ayarlanmalıdır? Varsayılan değer hangisidir?

Cevabın: _____

2. Markdown formatlı bir metin göstermek istiyorsun. Hem `textFormat` ayarını hem de örnek bir `setText` çağrısını yaz.

Cevabın: _____

Cevap Anahtarı

1. `Qt::RichText`'e ayarlanmalıdır. Varsayılan `Qt::AutoText`'tir; bu mod metni inceleyerek HTML etiketleri görürse otomatik olarak Rich Text'e geçer, görmezse Plain Text gibi davranır.
2. `label->setTextFormat(Qt::MarkdownText);`
`label->setText("**Kalin** ve *italik* metin");`

1.3 QPushButton

QObject → QWidget → QAbstractButton → QPushButton

Kullanıcının tıklayabileceği en temel buton. Genellikle bir "aksiyonu başlatır": kaydet, iptal, gönder. OBYS'deki "Not Ekle" veya "Kaydet" butonları birer QPushButton'dur. İki durumlu (`checkable`) yapılabilir; bir grup butonla birlikte toggle davranışı sergileyebilir.

Önemli Properties

Property	Tip	Açıklama
text	QString	Butonun üzerindeki yazı
icon	QIcon	Buton simgesi (opsiyonel)
checkable	bool	İki durumlu açılıp kapanır mı
checked	bool	Şu anki durumu (checkable ise)
enabled	bool	Tıklanabilir mi
default	bool	Dialog'da Enter'a varsayılan mı
flat	bool	Düz görünüm (kenarlık yok)

Signals

- `clicked(bool checked = false)` – Buton tıklandığında yayınlanır
- `pressed()` – Fare tuşu basıldığında (henüz bırakılmadan)
- `released()` – Fare tuşu bırakıldığında
- `toggled(bool checked)` – Checkable butonda durum değiştiğinde

Slots

- `click()` – Programatik tıklama
- `toggle()` – Durumu çevirir
- `setChecked(bool)` – Durumu ayarlar
- `setText(const QString&)` – Yazıyı değiştirir

C++ - Mini ornek: OBYS Kaydet butonu

```
auto* kaydet = new QPushButton(tr("Kaydet"), this);
kaydet->setIcon(QIcon(":/icons/file-save.svg"));
kaydet->setShortcut(QKeySequence::Save);
connect(kaydet, &QPushButton::clicked,
        this,   &MainWindow::dosyaKaydet);
```

Alıştırma

1. `clicked(bool checked)` sinyalinde `checked` parametresi hangi durumda anlamlıdır? (*ipucu: bir property ile ilgili*)

Cevabın: _____

2. Bir QPushButton'u iki durumlu (toggle) hale getirmek için hangi property `true` yapılmalı?

Cevabın: _____

Cevap Anahtarı

1. Buton `checkable` property'si `true` ise `checked` parametresi butonun yeni durumunu bildirir. Aksi hâlde (varsayılan buton) parametre her zaman `false` gelir ve kullanılmaz.
2. `checkable` property'si `true` yapılmalı: `btn->setCheckable(true);`

1.4 QToolButton

QObject → QWidget → QAbstractButton → QToolButton

Genellikle araç çubuklarında (QToolBar) kullanılan kompakt buton. Varsayılanda yalnızca bir simge gösterir; isteğe bağlı olarak simge + metin de gösterebilir. Bir QAction'a bağlanarak otomatik metin/ikon/kısayol/tooltip alabilir, hatta bir açılır menü (popup) sunabilir. OBYS'deki "Yeni Belge" butonunun yanındaki küçük ok bir QToolButton popup'ıdır.

Önemli Properties

Property	Tip	Açıklama
toolButtonStyle	Qt::ToolButtonStyle	Sadece ikon / yazı / ikon+yazı
popupMode	QToolButton::ToolButtonPopupMode	Menü açma davranışı
arrowType	Qt::ArrowType	Ok yönü (Up/Down/Left/Right)
autoRaise	bool	Üstüne gelince kabarma efekti
defaultAction	QAction*	Bağlı varsayılan aksiyon

Signals

- `triggered(QAction* action)` – Bir menü öğesi tetiklendi
- `clicked(bool checked)` – Buton tıklandı (QAbstractButton'dan)

Slots

- `setDefaultAction(QAction*)` – Varsayılan aksiyonu ata
- `showMenu()` – Bağlı popup menüyü göster
- `setToolButtonStyle(Qt::ToolButtonStyle)` – Stili değiştir

C++ - Mini ornek: Yeni belge tool butonu + popup menu

```cpp
auto* yeniBtn = new QToolButton(this);
yeniBtn->setIcon(QIcon(":/icons/document-new.svg"));
yeniBtn->setToolButtonStyle(Qt::ToolButtonTextBesideIcon);
yeniBtn->setText(tr("Yeni"));

auto* menu = new QMenu(yeniBtn);
menu->addAction(tr("Bos Belge"));
menu->addAction(tr("Sablondan"));
yeniBtn->setMenu(menu);
yeniBtn->setPopupMode(QToolButton::MenuButtonPopup);
```

Alıştırma

1. `toolButtonStyle` property'sinin alabileceği değerlerden en az üç tanesini yaz ve farklarını kısaca açıkla.

Cevabın: _____

2. `popupMode` için `DelayedPopup`, `MenuButtonPopup` ve `InstantPopup` arasındaki fark nedir?

Cevabın: _____

Cevap Anahtarı

1. `Qt::ToolButtonIconOnly` (sadece ikon), `Qt::ToolButtonTextOnly` (sadece yazı), `Qt::ToolButtonTextBesideIcon` (ikonun yanında yazı), `Qt::ToolButtonTextUnderIcon` (ikonun altında yazı), `Qt::ToolButtonFollowStyle` (sistem stilini takip eder).
2. `DelayedPopup`: Butona basılı tutunca menü açılır, normal tıklama varsayılan aksiyonu çalıştırır. `MenuButtonPopup`: Buton iki bölgeye ayrılır; ana kısım aksiyonu, küçük ok kısmı menüyü açar. `InstantPopup`: Tıklama anında menü açılır, aksiyon çalıştırılmaz.

1.5 QCommandLinkButton

QObject → QWidget → QAbstractButton → QPushButton → QCommandLinkButton

Windows Vista ile gelen "komut bağlantı" butonunun Qt karşılığı. Klasik bir QPushButton'a göre daha büyük, başlık + tek satır açıklama metni içerir ve genellikle "ileri" okuyla birlikte gösterilir. Wizard ekranlarında, karar diyaloglarında, hoş geldin sayfalarında ideal seçimdir. OBYS'de "Yeni Oturum Aç" veya "Mevcut Oturuma Devam Et" gibi eşdeğer seçenekleri sunarken kullanılır. QPushButton'dan türediği için tüm sinyalleri miras alır.

Önemli Properties

Property	Tip	Açıklama
text	QString	Üstteki büyük başlık
description	QString	Başlığın altındaki açıklama satırı
icon	QIcon	Sol taraftaki ikon (varsayılan ok)

Signals

- `clicked(bool checked = false)` – QPushButton'dan miras
- `pressed()`, `released()`, `toggled(bool)` – QAbstractButton'dan miras

Slots

- `setDescription(const QString&)` – Açıklamayı değiştir
- `setText(const QString&)` – Başlığı değiştir
- `setIcon(const QIcon&)` – İkonu değiştir

C++ - Mini ornek: OBYS oturum secimi

```
auto* yeniOturum = new QCommandLinkButton(
    tr("Yeni Oturum Baslat"),
    tr("Bu donem icin yeni bir akademik oturum olustur."),
    this);

connect(yeniOturum, &QCommandLinkButton::clicked,
        this,       &OturumWizard::yeniOturumAc);
```

Alıştırma

1. QCommandLinkButton'un sıradan bir QPushButton'dan görsel olarak en büyük üç farkı nedir?

Cevabın: _____

2. `description` metni, başlık metnine göre nasıl bir font/stil ile render edilir? (Boyut, kalınlık, renk)

Cevabın: _____

Cevap Anahtarı

1. (a) Çok daha büyük ve dikdörtgen bir tıklama alanı, (b) iki satırlı yapı: üstte büyük başlık, altta küçük açıklama, (c) sol tarafta varsayılan olarak gelen "→" okunu otomatik gösterir.
2. `description` metni başlığa göre daha küçük puntoda, normal kalınlıkta (başlık genelde bold) ve daha açık renkte (gri tonu) render edilir. Amaç başlığı vurgulamak ve açıklamayı ikincil tutmaktır.

1.6 QFrame

QObject → QWidget → QFrame

Çerçeveli widget'ların ortak temel sınıfı. QLabel, QGroupBox, QAbstractScrollArea (ve dolayısıyla QListView, QTextEdit) gibi pek çok widget bu sınıftan türemiştir. Doğrudan kullanıldığında ise iki ana amaca hizmet eder: (1) içine başka widget'lar yerleştirip görsel olarak gruplamak (kart/panel görünümü), (2) bir yatay/dikey ayırıcı çizgi (`HLine`, `VLine`) olarak çalışmak. OBYS form sayfalarındaki bölümleri ayıran ince gri çizgiler birer QFrame'dir.

Önemli Properties

Property	Tip	Açıklama
frameShape	QFrame::Shape	NoFrame / Box / Panel / HLine / VLine / WinPanel / StyledPanel
frameShadow	QFrame::Shadow	Plain / Raised / Sunken
lineWidth	int	Çerçeve çizgi kalınlığı (px)
midLineWidth	int	İki çizgi arası orta hat (Box stili)
frameRect	QRect	Çerçevenin geometrisi

Signals

- QFrame'in kendine özgü sinyali yoktur; QWidget sinyalleri kullanılır.

Slots

- `setFrameShape(QFrame::Shape)` – Şekli ayarla
- `setFrameShadow(QFrame::Shadow)` – Gölgeyi ayarla
- `setLineWidth(int)` – Kalınlığı ayarla
- `setFrameStyle(int)` – Şekil + gölgeyi tek seferde ayarla (`Shape | Shadow`)

C++ - Mini ornek: Yatay ayirici cizgi

```
auto* line = new QFrame(this);
line->setFrameShape(QFrame::HLine);
line->setFrameShadow(QFrame::Sunken);
line->setLineWidth(1);
formLayout->addRow(line);
```

Alıştırma

1. `QFrame::HLine` ile `QFrame::VLine` arasındaki fark nedir? Hangisini bir QHBoxLayout içinde iki buton arasına koyarsın?

Cevabın: _____

2. Bir QFrame'in çerçevesini hem `Box` şekli hem de `Sunken` gölgesi ile aynı anda ayarlamak için hangi tek slot çağrısını yazarsın?

Cevabın: _____

Cevap Anahtarı

1. `HLine` yatay (geniş, ince), `VLine` dikey (dar, uzun) bir ayırıcı çizgi çizer. QHBoxLayout içinde soldan sağa giden butonların arasına dik bir ayırıcı koymak istersek `VLine` kullanırız.
2. `frame->setFrameStyle(QFrame::Box | QFrame::Sunken);`
Bu çağrı şekil ve gölgeyi tek bit-OR ifadesiyle birlikte ayarlar.

Kategori Bitiş Quizi

5 Soruluk Karma Quiz

1. QWidget Qt hiyerarşisinde hangi sınıfın doğrudan türevidir?

---

2. (Eşleştirme) Aşağıdaki sinyal-slot eşleştirmesini yap. Her sinyal için makul bir slot/davranış öner:

`QPushButton::clicked` ?
`QLabel::linkActivated` ?
`QAbstractButton::toggled` ?

3. (Kod okuma) Aşağıdaki kodda `m_sayac` değişkenine ne olur?

C++

```cpp
QPushButton* btn = new QPushButton("Artir");
int m_sayac = 0;
connect(btn, &QPushButton::clicked, [&]{ ++m_sayac; });
btn->click();
btn->click();
btn->click();
```

---

4. QFrame'in çerçeve stilini kontrol eden iki property hangisidir?

---

5. (Mikro kod) Bir QLabel'a kalın siyah bir kenarlık eklemek için gereken `setFrameStyle` ve `setLineWidth` çağrılarını yaz:

Cevap Anahtarı

1. QObject'in doğrudan türevidir (`QObject` → `QWidget`).
2. Tipik bağlantılar: `clicked` → bir aksiyon slot'u (örn. `dosyaKaydet`); `linkActivated` → URL açıcı slot (`QDesktopServices::openUrl`); `toggled` → durum güncelleyici bir slot (örn. ilgili widget'ı göster/gizle).
3. `m_sayac` değeri 3 olur. Her `btn->click()` programatik tıklama `clicked` sinyalini yayınlar ve lambda her seferinde sayacı bir artırır.
4. `frameShape` ve `frameShadow`.
5. `label->setFrameStyle(QFrame::Box | QFrame::Plain);`
`label->setLineWidth(2);`
`label->setStyleSheet("border-color: black;");`

2 Giriş Alanları

Kullanıcı verisini toplayan widget'lar: kısa metin (QLineEdit), uzun metin (QTextEdit, QPlainTextEdit), sayı (QSpinBox, QDoubleSpinBox), kaydırıcı tabanlı değerler (QSlider, QDial, QScrollBar) ve klavye kısayolu girişi (QKeySequenceEdit). OBYS'deki ders notu giriş formunda (*(bkz. kitap)*) bu widget'ların birçoğunu zaten kullandık.

2.1 QLineEdit

QObject → QWidget → QLineEdit

Tek satır metin girişi için kullanılan en temel "girdi" widget'ıdır. Şifre modu (`echoMode`), karakter maskesi (`inputMask`) ve QValidator ile doğrulama destekler. OBYS giriş formunda öğrenci numarası, e-posta veya arama kutusu için ideal seçimdir. `returnPressed` sinyali sayesinde Enter tuşuyla tetiklenen hızlı arama akışları kurulabilir.

Önemli Properties

Property	Tip	Açıklama
text	QString	Mevcut metin
placeholderText	QString	Boşken görünen ipucu
maxLength	int	Maksimum karakter sayısı
echoMode	EchoMode	Normal/Password/NoEcho
readOnly	bool	Salt-okunur mod
inputMask	QString	Karakter maskesi
alignment	Qt::Alignment	Metin hizalaması
clearButtonEnabled	bool	Sağda temizleme butonu

Signals

- `textChanged(const QString&)` – Metin her değiştiğinde (programlı dahil)
- `textEdited(const QString&)` – Sadece kullanıcı düzenlediğinde
- `editingFinished()` – Enter veya odak kaybı
- `returnPressed()` – Enter tuşuna basıldı
- `cursorPositionChanged(int, int)` – İmleç konumu değişti

Slots

- `setText(const QString&)` – Metni değiştir
- `clear()` – Temizle
- `selectAll()`, `copy()`, `cut()`, `paste()`
- `undo()`, `redo()`

C++ - Mini örnek: OBYS öğrenci numarası girişi

```
auto* numara = new QLineEdit(this);
numara->setPlaceholderText(tr("Ornegin: 20240001"));
numara->setMaxLength(8);
numara->setClearButtonEnabled(true);
```

```
5
6  // Sadece rakam kabul et
7  auto* val = new QRegularExpressionValidator(
8      QRegularExpression("\\d{0,8}"), this);
9  numara->setValidator(val);
10
11 connect(numara, &QLineEdit::returnPressed,
12         this, &OgrenciFormu::ogrenciAra);
```

Alıştırma

1. `textChanged` ve `textEdited` sinyallerinin arasındaki temel fark nedir? Hangisi `setText()` çağrısında da yayınlanır?

Cevabın: _____

2. `placeholderText` ne zaman görünür ekranda? Bir şifre alanı için `echoMode` hangi değere ayarlanmalıdır?

Cevabın: _____

Cevap Anahtarı

1. `textChanged` hem programlı (`setText`) hem de kullanıcı düzenlemelerinde yayınlanır. `textEdited` yalnızca kullanıcı klavye/mouse ile düzenleme yaptığında yayınlanır. Sonsuz döngülerden kaçınmak için programlı güncellemelerde `textEdited` tercih edilir.
2. `placeholderText`, `text` boş olduğunda ve widget odakta değilken (Qt 5.12+ odakta da) gösterilir. Şifre alanı için `echoMode` `QLineEdit::Password` olmalıdır (yıldız/nokta gösterir).

2.2 QTextEdit

QObject → QWidget → QFrame → QAbstractScrollArea → QTextEdit

Çok satırlı *rich text* (zengin metin) editörüdür. HTML alt kümesi, Markdown ve düz metin destekler; içinde QTextDocument nesnesi tutar. OBYS'de ders açıklaması, ders notu yorumu veya "Hakkında" diyaloğunda biçimlendirilmiş metin göstermek için kullanılır. Raporlama modülünde QTextDocument üzerinden PDF dışa aktarımı bu widget ile aynı belgeyi paylaşır.

Önemli Properties

Property	Tip	Açıklama
plainText	QString	Düz metin içeriği
html	QString	HTML içeriği
readOnly	bool	Salt-okunur mod
acceptRichText	bool	Yapıştırmada zengin metin
textInteractionFlags	Qt::TextInteraction	Etkileşim seviyesi
tabStopDistance	qreal	Tab genişliği (px)
placeholderText	QString	Boşken ipucu

Signals

- `textChanged()` – İçerik değişti
- `cursorPositionChanged()` – İmleç hareket etti
- `selectionChanged()` – Seçim değişti
- `copyAvailable(bool)` – Kopyalanabilir seçim var

Slots

- `setPlainText(const QString&)` – Düz metin ata
- `setHtml(const QString&)` – HTML ata
- `setMarkdown(const QString&)` – Markdown ata
- `append(const QString&)` – Sona ekle
- `clear()`, `undo()`, `redo()`

C++ - Mini örnek: OBYS Hakkında metni

```
auto* hakkinda = new QTextEdit(this);
hakkinda->setReadOnly(true);
hakkinda->setHtml(tr(
    "<h2>OBYS v1.0</h2>"
    "<p>Ogrenci Bilgi Yonetim Sistemi</p>"
    "<p><b>Gelistirici:</b> KTU Bilgisayar Bilimleri</p>"
    "<p><a href='https://ktu.edu.tr'>ktu.edu.tr</a></p>"));
```

Alıştırma

1. `setHtml()` ve `setPlainText()` çağrıları arasındaki fark nedir? `<b>Merhaba</b>` metnini her iki yöntemle verirseniz ekranda ne görünür?

Cevabın: _____

2. `acceptRichText` özelliği `false` yapılırsa ne değişir? Hangi senaryoda bunu kapatmak isteriz?

Cevabın: _____

Cevap Anahtarı

1. `setHtml()` metni HTML olarak yorumlar; `<b>` etiketleri kalın metne dönüşür ve ekranda **Merhaba** görünür. `setPlainText()` ise her karakteri olduğu gibi gösterir; ekranda `<b>Merhaba</b>` yazısı görünür.
2. `acceptRichText false` yapıldığında kullanıcı zengin metin (örn. tarayıcıdan kopyalanmış biçimli yazı) yapıştırsa bile widget yalnızca düz metin olarak alır. Bir form alanında stil bütünlüğünü korumak için bu özellik kapatılır.

2.3 QPlainTextEdit

QObject → QWidget → QFrame → QAbstractScrollArea → QPlainTextEdit

QTextEdit'in performans için optimize edilmiş düz metin versiyonudur. Stil/biçim taşımadığı için çok büyük belgelerde (binlerce satırlık log dosyası, kaynak kod) çok daha hızlıdır. OBYS'de oturum açma denemesi, ağ trafiği ya da arka plan görev log panelleri için doğru seçimdir. `maximumBlockCount` sayesinde otomatik olarak eski satırları atan halka tampon davranışı kurulabilir.

Önemli Properties

Property	Tip	Açıklama
plainText	QString	Tüm metin
readOnly	bool	Salt-okunur
maximumBlockCount	int	Maks. paragraf (0 = sınırsız)
lineWrapMode	LineWrapMode	NoWrap / WidgetWidth
placeholderText	QString	Boşken ipucu
tabStopDistance	qreal	Tab genişliği

Signals

- `textChanged()` – İçerik değişti
- `cursorPositionChanged()` – İmleç hareket etti
- `blockCountChanged(int)` – Paragraf sayısı değişti
- `updateRequest(QRect, int)` – Görünüm güncellemesi

Slots

- `appendPlainText(const QString&)` – Sona satır ekle
- `setPlainText(const QString&)` – Tüm metni değiştir
- `clear()` – Temizle
- `centerCursor()` – İmleci görünür yap

C++ - Mini örnek: OBYS log paneli (son 100 satır)

```cpp
auto* log = new QPlainTextEdit(this);
log->setReadOnly(true);
log->setMaximumBlockCount(100);  // halka tampon
log->setLineWrapMode(QPlainTextEdit::NoWrap);

// Bir log satiri ekleme
auto satirEkle = [log](const QString& msg) {
    auto zaman = QTime::currentTime().toString("HH:mm:ss");
    log->appendPlainText(QString("[%1] %2").arg(zaman, msg));
};
satirEkle(tr("Sunucuya baglanildi."));
```

Alıştırma

1. `maximumBlockCount` property'si nedir, ne işe yarar? Bir log paneli için neden kritiktir?

Cevabın: _____

2. QPlainTextEdit, QTextEdit'e göre hangi senaryolarda tercih edilmelidir? En az iki örnek ver.

Cevabın: _____

Cevap Anahtarı

1. `maximumBlockCount`, widget'ın içerebileceği maksimum paragraf (satır) sayısını belirler. Bu sınır aşıldığında en eski satırlar otomatik olarak silinir; halka tampon davranışı oluşur. Sürekli log akan paneller için bellek sızıntısını ve sonsuz büyümeyi engeller.
2. (a) Çok büyük metin/log dosyaları, (b) sözdizimi vurgulu kod editörleri (rich text gerekmeyen), (c) network/sistem log panelleri, (d) komut satırı çıktıları gibi düz metin tabanlı ve performans kritik senaryolarda QPlainTextEdit tercih edilir.

2.4 QSpinBox

QObject → QWidget → QAbstractSpinBox → QSpinBox

Belirli aralıkta tam sayı seçmek için kullanılan kontrol; üstünde yukarı/aşağı oklar bulunur. `minimum`/`maximum` sınırları sayesinde kullanıcının geçersiz değer girmesi mümkün değildir. OBYS'de kayıt yılı, kredi sayısı, kontenjan veya ders süresi (saat) gibi tam sayı alanları için ideal seçimdir. `prefix` ve `suffix` sayesinde değerin yanına birim eklenebilir (örn. "`6 kredi`").

Önemli Properties

Property	Tip	Açıklama
value	int	Mevcut değer
minimum	int	Alt sınır
maximum	int	Üst sınır
singleStep	int	Ok başına artış
prefix	QString	Değer öncesi metin
suffix	QString	Değer sonrası metin
wrapping	bool	Sınırda başa dön
specialValueText	QString	minimum için özel metin

Signals

- `valueChanged(int)` – Değer değişti
- `textChanged(const QString&)` – Metin temsili değişti
- `editingFinished()` – Düzenleme tamamlandı

Slots

- `setValue(int)` – Değeri ayarla
- `stepUp()` / `stepDown()` – Bir adım artır/azalt
- `clear()` – Temizle

C++ - Mini örnek: OBYS ders kredisi

```cpp
auto* kredi = new QSpinBox(this);
kredi->setRange(1, 8);
kredi->setValue(3);
kredi->setSuffix(tr(" kredi"));
kredi->setSpecialValueText(tr("Belirsiz"));  // value == 1 icin?
// Hayir: specialValueText sadece minimum'da gosterilir.
// Bu ornekte minimum = 1 oldugunda "Belirsiz" yazar.

connect(kredi, &QSpinBox::valueChanged,
        this, &DersFormu::krediDegisti);
```

Alıştırma

1. `specialValueText` property'si ne için kullanılır? Hangi değer geldiğinde devreye girer?

Cevabın: _____

2. `wrapping` property'si `true` ise kullanıcı `maximum`'a ulaşıp tekrar yukarı oka basarsa ne olur?

Cevabın: _____

Cevap Anahtarı

1. `specialValueText`, değer tam olarak `minimum` değerine eşit olduğunda sayı yerine gösterilen metindir. Genellikle "Sınırsız", "Otomatik", "Belirsiz" gibi özel anlamlar için kullanılır.
2. `wrapping == true` iken kullanıcı `maximum` değerinden bir adım daha artırırsa değer `minimum`'a sarmalanır (başa döner). `false` ise sınırda durur.

2.5 QDoubleSpinBox

QObject → QWidget → QAbstractSpinBox → QDoubleSpinBox

QSpinBox'ın ondalık (double) versiyonudur. `decimals` özelliği ile gösterilecek ondalık basamak sayısı ayarlanır. OBYS'de sınav notu (0.0–100.0), GANO (0.00–4.00) veya ders ücreti gibi kesirli sayı alanları için kullanılır. Hassasiyet gerektirmeyen yüzde gösterimi için tek ondalık (`decimals=1`) yeterli olurken, GANO için iki ondalık tercih edilir.

Önemli Properties

Property	Tip	Açıklama
value	double	Mevcut değer
minimum	double	Alt sınır
maximum	double	Üst sınır
singleStep	double	Ok başına artış
decimals	int	Ondalık basamak sayısı
prefix	QString	Değer öncesi metin
suffix	QString	Değer sonrası metin

Signals

- `valueChanged(double)` – Sayısal değer değişti
- `textChanged(const QString&)` – Metin temsili değişti

Slots

- `setValue(double)` – Değeri ayarla
- `setDecimals(int)` – Ondalık basamak ayarla
- `stepUp()` / `stepDown()`

C++ - Mini örnek: OBYS sınav notu girişi

```cpp
auto* not_ = new QDoubleSpinBox(this);
not_->setRange(0.0, 100.0);
not_->setDecimals(1);
not_->setSingleStep(0.5);
not_->setSuffix(tr(" puan"));
not_->setValue(75.0);

connect(not_, &QDoubleSpinBox::valueChanged,
        this, &NotFormu::notHesapla);
```

Alıştırma

1. `valueChanged(int)` ve `valueChanged(double)` sinyalleri QDoubleSpinBox'ta birlikte bulunur. Connect yaparken eski Qt sürümlerinde `QOverload<double>::of(...)` neden gerekli olabilir?

Cevabın: _____

2. `decimals` 2'den 1'e indirilirse var olan değer `87.55` nasıl görünür? İçerideki `value` değeri değişir mi?

Cevabın: _____

Cevap Anahtarı

1. Aynı isimde fakat farklı parametre tipinde aşırı yüklenmiş (overloaded) sinyal olduğu için derleyici hangisini bağlayacağını seçemez. `QOverload<double>::of(&QDoubleSpinBox::valueChanged)` yazarak doğru aşırı yüklemeyi seçeriz. Qt 5.7+ ile fonksiyon işaretçisi syntax'ı geliştirilse de C++ aşırı yükleme belirsizliği hâlâ çözülmesi gereken bir konudur.
2. Görünen metin `87.6` olur (yuvarlama). `value()` ise önceki tam değeri (`87.55`) korumaya devam edebilir, ancak sonraki bir `setValue` veya kullanıcı düzenlemesi ekrandaki hassasiyetle yeniden yuvarlanır.

2.6 QSlider

QObject → QWidget → QAbstractSlider → QSlider

Yatay veya dikey bir kaydırıcı (slider). Kullanıcının bir aralıktan süreklilik hissiyle değer seçmesini sağlar. OBYS'de ders not ağırlığı (%0–%100), filtre eşiği veya sınav süre çubuğu gibi göreceli değerler için uygundur. Tüm temel davranışı QAbstractSlider'dan miras alır; QSlider ise yalnızca çizimle (tick işaretleri, yön) ilgilenir.

Önemli Properties

Property	Tip	Açıklama
value	int	Mevcut değer
minimum	int	Alt sınır
maximum	int	Üst sınır
orientation	Qt::Orientation	Horizontal / Vertical
singleStep	int	Ok tuşu adımı
pageStep	int	PageUp/PageDown adımı
tickPosition	TickPosition	NoTicks / Above / Below / Both
tickInterval	int	Tick aralığı
tracking	bool	Sürüklerken sinyal yay

Signals

- `valueChanged(int)` – Değer her değiştiğinde
- `sliderMoved(int)` – Sürükleme sırasında (yalnız kullanıcı)
- `sliderPressed()` / `sliderReleased()` – Mouse tutuş
- `rangeChanged(int, int)` – Aralık değişti

Slots

- `setValue(int)` – Değeri ayarla
- `setOrientation(Qt::Orientation)`
- `setRange(int, int)`

C++ - Mini örnek: OBYS not ağırlığı sürgüsü

```
auto* agirlik = new QSlider(Qt::Horizontal, this);
agirlik->setRange(0, 100);
agirlik->setValue(40);
agirlik->setTickPosition(QSlider::TicksBelow);
agirlik->setTickInterval(10);
agirlik->setSingleStep(5);
```

```
7
8  connect(agirlik, &QSlider::valueChanged,
9          this, [](int v) {
10             qDebug() << "Vize agirligi: %" << v;
11         });
```

Alıştırma

1. `tickPosition` özelliğinin alabileceği değerleri listele. Görsel olarak ne fark yaratırlar?

Cevabın: _____

2. `valueChanged(int)` ile `sliderMoved(int)` sinyalleri arasındaki fark nedir? Programlı `setValue(50)` çağrısında hangisi yayınlanır?

Cevabın: _____

Cevap Anahtarı

1. `QSlider::NoTicks` (hiç çizgi yok), `TicksAbove` / `TicksLeft` (üstte/solda çizgiler), `TicksBelow` / `TicksRight` (altta/sağda), `TicksBothSides` (her iki yanda). Çizgiler yalnızca görseldir, değer aralığını değiştirmezler.
2. `valueChanged` hem programlı (`setValue`) hem de kullanıcı kaynaklı tüm değişikliklerde yayınlanır. `sliderMoved` yalnızca kullanıcı kolu sürüklerken yayınlanır; `setValue(50)` çağrısında `valueChanged` yayınlanır, `sliderMoved` yayınlanmaz.

2.7 QDial

QObject → QWidget → QAbstractSlider → QDial

Klasik analog potansiyometre (ses mikser düğmesi) görünümünde döner bir kontroldür. QSlider ile aynı tabandan (QAbstractSlider) türediği için aynı API'yi kullanır. Açı seçimi (0–360), ses seviyesi mikseri veya ekolayzır gibi kullanım alanları vardır. OBYS'de gündelik form widget'ı olarak nadir görünse de bir grafik düzenleyicide eksen açısı ya da renk tonu seçimi için kullanılabilir.

Önemli Properties

Property	Tip	Açıklama
value	int	Mevcut değer (miras)
minimum	int	Alt sınır (miras)
maximum	int	Üst sınır (miras)
notchesVisible	bool	Çentiklerin gösterilmesi
notchTarget	qreal	Çentikler arası hedef piksel
wrapping	bool	360'ta başa dönsün mü

Signals

- QAbstractSlider'dan miras: `valueChanged(int)`, `sliderMoved(int)`, `sliderPressed()`, `sliderReleased()`

Slots

- `setValue(int)` – Değeri ata
- `setNotchesVisible(bool)` – Çentikleri göster/gizle
- `setWrapping(bool)` – Sarma davranışı

C++ - Mini örnek: 0-360 açı seçici

```
auto* aci = new QDial(this);
aci->setRange(0, 359);
aci->setWrapping(true);          // 359 -> 0 surekli donus
aci->setNotchesVisible(true);
aci->setNotchTarget(8.0);
aci->setValue(45);

connect(aci, &QDial::valueChanged,
        this, [](int v) { qDebug() << "Aci:" << v; });
```

Alıştırma

1. `wrapping` özelliği `true` olduğunda fiziksel olarak ne fark yaratır? Hangi senaryolarda `true` yapmak mantıklıdır?

Cevabın: _____

2. `notchesVisible` ve `notchTarget` ne işe yarar? `notchTarget`'ın birimi nedir?

Cevabın: _____

Cevap Anahtarı

1. `wrapping == true` iken QDial 360 derece sürekli dönebilir; `maximum`'u geçtiğinde değer `minimum`'a sarmalanır. `false` iken üst sınırda durur ve dönmez. Açı seçimi, hue (renk tonu) veya analog ses pan kontrolü gibi döngüsel değerler için `true` mantıklıdır.
2. `notchesVisible`, dial'ın çevresinde çentik (işaret) çizgilerinin gösterilip gösterilmeyeceğini belirler. `notchTarget` ise iki çentik arasında bırakılmak istenen hedef piksel mesafesidir; Qt buna göre çentik sayısını otomatik hesaplar.

2.8 QScrollBar

QObject → QWidget → QAbstractSlider → QScrollBar

Kaydırma çubuğu. Genellikle doğrudan kullanılmaz; QAbstractScrollArea'dan türeyen widget'lar (QTextEdit, QListView, QTableView, QScrollArea) tarafından otomatik oluşturulur ve yönetilir. Manuel kullanım, özelleştirilmiş çizim alanları (QGraphicsView dışı) veya senkronize iki view arasında ortak kaydırma istendiğinde gerekir. QSlider ile aynı tabandan türediği için API'leri benzerdir.

Önemli Properties

Property	Tip	Açıklama
value	int	Mevcut konum (miras)
minimum	int	Alt sınır (miras)
maximum	int	Üst sınır (miras)
pageStep	int	Boş alana tıklama adımı
singleStep	int	Ok adımı
orientation	Qt::Orientation	Horizontal / Vertical

Signals

- `valueChanged(int)` – Konum değişti
- `sliderMoved(int)` – Kullanıcı tutuyor
- `actionTriggered(int)` – Ok / sayfa / tutuş eylemi

Slots

- `setValue(int)`, `setRange(int, int)`
- `setPageStep(int)`, `setSingleStep(int)`

C++ - Mini örnek: İki view'ı aynı anda kaydırma

```
auto* viewSol = new QListView(this);
auto* viewSag = new QListView(this);

// Sol view'in dikey scroll'unu sag view'e bagla
connect(viewSol->verticalScrollBar(), &QScrollBar::valueChanged,
        viewSag->verticalScrollBar(), &QScrollBar::setValue);
connect(viewSag->verticalScrollBar(), &QScrollBar::valueChanged,
        viewSol->verticalScrollBar(), &QScrollBar::setValue);
```

Alıştırma

1. QScrollBar'ı manuel oluşturmamız gereken bir senaryo örneği ver. Çoğunlukla neden manuel oluşturmaya gerek yoktur?

Cevabın: _____

2. `pageStep` ve `singleStep` arasındaki fark nedir? Hangi kullanıcı eylemleri hangisini tetikler?

Cevabın: _____

Cevap Anahtarı

1. İki ayrı görünüm arasında senkronize kaydırma kurmak, özel çizim yapan bir widget'ın kendi scroll alanını yönetmesi veya non-standart bir kaydırma davranışı (örn. mıknatıslı yapışma) istendiğinde manuel QScrollBar gerekir. Çoğunlukla QAbstractScrollArea türevleri scroll bar'ları otomatik oluşturduğu için elle ele almaya gerek yoktur.
2. `singleStep`, ok tuşları veya scroll bar'ın ucundaki ok düğmelerine basıldığında yapılan küçük adımdır. `pageStep`, çubuğun boş alanına tıklandığında veya PageUp/PageDown tuşlarına basıldığında yapılan büyük adımdır.

2.9 QKeySequenceEdit

QObject → QWidget → QKeySequenceEdit

Kullanıcıdan klavye kısayolu kaydetmek için tasarlanmış özel girdi widget'ı. Kullanıcı tuşa bastığında widget bunu metne çevirir (örn. "Ctrl+Shift+S"). OBYS'nin "Ayarlar > Kısayollar" diyaloğunda her komut için (Kaydet, Yeni Öğrenci, Arama Aç...) kullanıcının kendi kısayolunu atamasını sağlar. İçinde QKeySequence nesnesi tutar; bu da QAction'a doğrudan atanabilir.

Önemli Properties

Property	Tip	Açıklama
keySequence	QKeySequence	Mevcut kısayol
maximumSequenceLength	int	Maks. tuş sayısı
finishingKeyCombinations	QList<QKeyCombination>	Düzenlemeyi bitiren tuşlar
clearButtonEnabled	bool	Temizle butonu

Signals

- `keySequenceChanged(const QKeySequence&)` – Yeni kısayol kaydedildi
- `editingFinished()` – Düzenleme bitti

Slots

- `setKeySequence(const QKeySequence&)` – Programlı ata
- `clear()` – Temizle

C++ - Mini örnek: OBYS Kaydet kısayolu

```
auto* kisayolEdit = new QKeySequenceEdit(this);
kisayolEdit->setKeySequence(QKeySequence::Save);  // Ctrl+S

connect(kisayolEdit, &QKeySequenceEdit::keySequenceChanged,
        this, [this](const QKeySequence& seq) {
            kaydetAction->setShortcut(seq);
            qDebug() << "Yeni kisayol:" << seq.toString();
        });
```

Alıştırma

1. `maximumSequenceLength`'in varsayılan değeri kaçtır? Bu sayı neyi sınırlar (tuş kombinasyonu mu, ardışık kombo mu)?

Cevabın: _____

2. İki farklı eyleme aynı kısayol atanmasını nasıl engellersin? QKeySequenceEdit tek başına bunu kontrol eder mi?

Cevabın: _____

Cevap Anahtarı

1. Varsayılan değer 4'tür. Bu, art arda kaydedilebilen tuş *kombinasyonu* sayısıdır (Emacs/Vim tarzı çoklu tuş dizileri). Yani Ctrl+K, Ctrl+S gibi 4'e kadar ardışık kombinasyon kaydedilebilir. Modifikatörler (Ctrl, Shift) tek bir kombinasyonun parçasıdır, ayrı tuş sayılmazlar.
2. QKeySequenceEdit sadece girişi kaydeder; çakışma kontrolü uygulamanın sorumluluğundadır. `keySequenceChanged` sinyalinde gelen sekansı tüm QAction'ların shortcut listesiyle karşılaştırır, çakışma varsa kullanıcıyı uyarır veya diğer eylemin kısayolunu temizlersiniz.

Kategori 02 Bitiş Quizi

Giriş alanları kategorisini tamamladın! Aşağıdaki 5 soru ile bilgilerini ölç. Cevaplar bu sayfanın altında (eğitmen versiyonunda) yer alır.

Bitiş Quizi – Giriş Alanları

1. QSpinBox'ta kullanıcının *hiçbir koşulda* sınırların dışına çıkamamasını garanti eden iki property hangileridir?

Cevabın: _____

2. QLineEdit'in `textChanged()` ve `textEdited()` sinyalleri arasındaki temel fark nedir? Programlı `setText()` çağrısında hangisi yayınlanır?

Cevabın: _____

3. (Kod okuma) Aşağıdaki kodun çalıştırılması sonucunda `qDebug()` çıktısı ne olur?

C++ - Kod parcasi

```
auto* sp = new QDoubleSpinBox;
sp->setRange(0.0, 100.0);
sp->setDecimals(2);
sp->setValue(87.5);
qDebug() << sp->value();
```

Cevabın: _____

4. QPlainTextEdit hangi senaryolarda QTextEdit'e tercih edilmelidir? En az iki neden ve bir somut kullanım örneği ver.

Cevabın: _____

5. (Mikro kod) Bir QSlider'ı 0–255 aralığında, alt tarafında her 25 birimde bir tick işareti olacak ve tek adım 5 olacak şekilde auto'lu tek bir kod bloğunda hazırla.

Cevabın: _____

Bitiş Quizi – Cevap Anahtarı

1. `minimum` ve `maximum` property'leri. QSpinBox bu sınırların dışındaki değerleri otomatik olarak sınırlar; klavyeden geçersiz bir sayı girilse bile odak kaybında en yakın geçerli değere yuvarlanır.
2. `textChanged` hem programlı hem kullanıcı kaynaklı tüm değişikliklerde yayınlanır. `textEdited` yalnızca kullanıcı klavye/mouse ile düzenlediğinde yayınlanır. `setText()` çağrısında yalnızca `textChanged` yayınlanır, `textEdited` yayınlanmaz.
3. Çıktı: `87.5` (veya tipik olarak `87.5`). Değer 87.5 olarak içeride saklanır; `decimals=2` sadece ekranda gösterimi etkiler, `value()` hâlâ double döner ve değer değişmez. (Ekranda `87.50` olarak görünür.)
4. Tercih nedenleri: (a) Çok büyük metinlerde belirgin performans avantajı, (b) rich-text/HTML ek yükü olmaması nedeniyle düşük bellek kullanımı, (c) `maximumBlockCount` ile otomatik halka tampon. Somut örnek: bir log paneli, network trafik göstericisi, basit kod editörü veya komut satırı çıktı paneli.
5.

C++ - Cevap

```
auto* s = new QSlider(Qt::Horizontal, this);
s->setRange(0, 255);
s->setTickPosition(QSlider::TicksBelow);
s->setTickInterval(25);
s->setSingleStep(5);
```

3 Seçim ve Tercih

Kullanıcıya seçenek sunan widget'lar: açık/kapalı (QCheckBox), çoklu seçenekten tek seçim (QRadioButton + QButtonGroup) ve bir listeden tek seçim (QComboBox, QFontComboBox). OBYS form (*(bkz. kitap)*) bu widget'ların bir kısmını zaten kullandı.

3.1 QCheckBox

QObject → QWidget → QAbstractButton → QCheckBox

Açık/kapalı (boolean) seçim sunan kutucuk. "Beni hatırla", "E-posta bildirimleri al", "Otomatik kaydet" gibi birbirinden bağımsız seçenekler için idealdir. İsteğe bağlı olarak *üç durumlu* (tri-state) yapılabilir; bu mod "hepsini seç / hiçbirini seçme / kısmen seçili" senaryolarında işe yarar.

Önemli Properties

Property	Tip	Açıklama
text	QString	Kutunun yanındaki etiket metni
checked	bool	Seçili mi? (iki durumlu mod)
tristate	bool	Üç durumlu modu açar
checkState	enum	Unchecked / PartiallyChecked / Checked

Signals

- `stateChanged(int state)` – durum değişti (üç durumlu için)
- `toggled(bool checked)` – seçim açıldı/kapandı
- `clicked(bool checked)` – kullanıcı tıkladı

Slots

- `setChecked(bool)`, `toggle()`
- `setCheckState(Qt::CheckState)`

C++ - Karanlık mod tercihi

```
auto* karanlikMod = new QCheckBox("Karanlik mod", this);
karanlikMod->setChecked(false);
connect(karanlikMod, &QCheckBox::toggled, this, [this](bool acik){
    qDebug() << (acik ? "Tema: koyu" : "Tema: aydinlik");
});
```

Alıştırma

1. `tristate` özelliği hangi senaryoda kullanılır? Bir örnek arayüz davranışı yaz.

Cevabın: _____

2. `stateChanged(int)` sinyalinin parametresi olan `int` hangi sabitleri alabilir? Üçünü de listele.

Cevabın: _____

Cevap Anahtarı

1. "Hepsini seç" tarzı ana kutu için. Alt seçeneklerin bir kısmı seçiliyse ana kutu `PartiallyChecked` (yarı dolu) görünür; hepsi seçiliyse `Checked`, hiçbiri seçili değilse `Unchecked` olur. Dosya yöneticisindeki klasör seçimi gibi.
2. `Qt::Unchecked` (0), `Qt::PartiallyChecked` (1), `Qt::Checked` (2). Üç durumlu mod kapalıyken yalnızca Unchecked ve Checked üretilir.

3.2 QRadioButton

QObject → QWidget → QAbstractButton → QRadioButton

"Birden fazla seçenek var ama yalnızca biri seçilebilir" senaryosu için kullanılır. Aynı parent widget altındaki tüm radyo butonları **otomatik olarak** birbirini dışlar; yani birini seçtiğinizde diğerleri kendiliğinden seçimi bırakır. Birden fazla bağımsız grup istiyorsanız her grubu farklı bir QGroupBox (veya farklı parent) içine koyarsınız ya da QButtonGroup kullanırsınız.

Önemli Properties

Property	Tip	Açıklama
text	QString	Etiket metni
checked	bool	Seçili mi?
autoExclusive	bool	Otomatik dışlama (varsayılan: true)

Signals

- `toggled(bool checked)` – seçim değişti
- `clicked(bool checked)`, `pressed()`

Slots

- `setChecked(bool)`

C++ - Tema seçimi

```cpp
auto* aydinlik = new QRadioButton("Aydinlik", this);
auto* karanlik = new QRadioButton("Karanlik",  this);
auto* sistem   = new QRadioButton("Sistem",    this);
sistem->setChecked(true);  // varsayilan

connect(karanlik, &QRadioButton::toggled, this, [](bool s){
    if (s) qDebug() << "Karanlik tema secildi";
});
```

Alıştırma

1. `autoExclusive` ne zaman `false` yapılır? Mantıklı bir kullanım örneği ver.

Cevabın: _____

2. İki farklı QGroupBox içindeki radyo butonları birbirini etkiler mi? Neden?

Cevabın: _____

Cevap Anahtarı

1. Aynı parent altında olmasına rağmen birbirinden bağımsız davranması istenen radyolar için. Pratikte nadirdir; genelde QButtonGroup ile gruplama tercih edilir, ama özel durumlarda (örneğin programatik olarak hepsini birden "seçili değil" yapmak istemek) kapatılabilir.
2. Hayır. Çünkü autoExclusive yalnızca *aynı parent widget* içindeki radyolar arasında çalışır. QGroupBox'lar ayrı parent oldukları için her grup kendi içinde bağımsız çalışır – bu zaten radyo butonlarının kategori ayırma için QGroupBox ile birlikte kullanılmasının ana nedenidir.

3.3 QButtonGroup

QObject → QButtonGroup (görsel değil)

QButtonGroup ekranda hiçbir şey çizmez! Soyut bir *grup yöneticisidir*: birden fazla butonu mantıksal olarak bir grup gibi yönetmenizi sağlar. En çok radyo butonlarına ID atayıp tek bir merkezden "hangi seçenek seçildi" sinyalini dinlemek için kullanılır. Düzene (layout) eklemenize gerek yoktur; sadece butonları `addButton()` ile gruba kaydedersiniz.

Önemli Properties

Property	Tip	Açıklama
exclusive	bool	Tek seçim modu (varsayılan: true)

Signals

- `idClicked(int id)` – ID'si bilinen buton tıklandı
- `idToggled(int id, bool checked)`
- `buttonClicked(QAbstractButton* btn)`

Slots / Metotlar

- `addButton(button, id)`, `removeButton(button)`
- `setId(button, id)`, `checkedId()`, `checkedButton()`

C++ - Dil seçimi grubu

```cpp
auto* grup = new QButtonGroup(this);
grup->addButton(new QRadioButton("Turkce",  this), 0);
grup->addButton(new QRadioButton("English", this), 1);
grup->addButton(new QRadioButton("Deutsch", this), 2);
grup->addButton(new QRadioButton("Francais",this), 3);

connect(grup, &QButtonGroup::idClicked, this, [](int id){
    qDebug() << "Secilen dil id:" << id;
});
```

Alıştırma

1. `exclusive` özelliğini `false` yapmak ne zaman anlamlıdır? Hangi widget türünü gruplarken kullanırsın?

Cevabın: _____

2. `buttonClicked` sinyalinde neden `int` yerine `QAbstractButton*` parametre olarak verilir? Avantajı nedir?

Cevabın: _____

Cevap Anahtarı

1. Birden fazla QCheckBox'u tek bir gruptan dinlemek istediğinde. Checkbox'lar bağımsız seçilebileceğinden grup exclusive olmamalıdır; ama yine de tek bir signal ile hangi checkbox değişti bilgisini almak istersin (örneğin OBYS'de "aldığım dersler" listesi).
2. Pointer ile butona doğrudan erişebilir, `text()`, `property()` gibi her özelliğine ulaşabilirsin. ID yaklaşımı ise sadece sayı verir; arayüz dışında saklanan ek bilgileri (ders kodu, kullanıcı verisi) okumak için butonun kendisine erişmek daha esnektir.

3.4 QComboBox

QObject → QWidget → QComboBox

Açılır menüden tek seçim sunan kompakt widget. Ekranda yer kaplamadan düzinelerce seçenek arasından birini seçmenizi sağlar. İsteğe bağlı olarak `editable` yapılırsa kullanıcı listede olmayan bir metni de yazabilir (autocomplete + serbest giriş). OBYS'de ders seçimi, not türü (vize/final/proje), dönem seçimi gibi yerlerde sıkça karşımıza çıkar.

Önemli Properties

Property	Tip	Açıklama
currentIndex	int	Seçili öğe indeksi (-1 yok demek)
currentText	QString	Seçili öğenin metni
count	int	Toplam öğe sayısı
editable	bool	Kullanıcı yazabilir mi?
maxCount	int	Maksimum öğe sınırı
insertPolicy	enum	Yeni metin nereye eklensin
sizeAdjustPolicy	enum	İçeriğe göre boyut ayarı

Signals

- `currentIndexChanged(int)` – programatik veya kullanıcı
- `currentTextChanged(const QString&)`
- `activated(int)` – yalnızca kullanıcı seçimi
- `editTextChanged(const QString&)` – editable modda

Slots

- `addItem`, `addItems`, `insertItem`
- `removeItem`, `clear`
- `setCurrentIndex`, `setCurrentText`

C++ - Ders seçici

```
auto* dersSec = new QComboBox(this);
dersSec->addItem("BILB1001 - Programlamaya Giris");
dersSec->addItem("BILB2000 - Yazilim Gelistirme II");
dersSec->addItem("BILB3010 - Veri Yapilari");

connect(dersSec, &QComboBox::currentIndexChanged,
        this, [dersSec](int idx){
    qDebug() << "Secilen:" << dersSec->itemText(idx);
});
```

Alıştırma

1. `currentIndexChanged` ile `activated` sinyali arasındaki temel fark nedir? Hangi durumda `activated` tercih edilir?

Cevabın: _____

2. `editable true` ise kullanıcının yazdığı metni hangi sinyal yakalar? Listeye otomatik eklenmesini hangi property kontrol eder?

Cevabın: _____

Cevap Anahtarı

1. `currentIndexChanged` hem kullanıcı seçimi hem de `setCurrentIndex()` gibi programatik değişikliklerde tetiklenir. `activated` ise yalnızca kullanıcı listeden seçim yaptığında çalışır. Programatik değişikliği kullanıcı eylemiymiş gibi yorumlamak istemediğinde `activated` daha güvenlidir.
2. `editTextChanged(const QString&)` sinyali yakalar. Otomatik liste ekleme davranışı ise `insertPolicy` property'si ile kontrol edilir (`NoInsert`, `InsertAtTop`, `InsertAtBottom`, `InsertAlphabetically`, vb.).

3.5 QFontComboBox

QObject → QWidget → QComboBox → QFontComboBox

QComboBox'tan türetilen, sistemde yüklü tüm font ailelerini otomatik olarak listeleyen özelleşmiş bir combo. Her öğe kendi fontuyla render edildiği için kullanıcı seçeceği fontu önizler. OBYS'de transkript çıktısı, raporlama veya tema editörü gibi yerlerde işe yarar. Kendinizin elle font listesi doldurmanıza gerek yoktur; widget bunu sizin için yapar.

Önemli Properties

Property	Tip	Açıklama
currentFont	QFont	Seçili font ailesi
writingSystem	enum	Hangi yazı sistemi (Latin, Arabic, ...)
fontFilters	flags	Hangi font türleri listelensin

Signals

- `currentFontChanged(const QFont&)` – yeni font seçildi

Slots

- `setCurrentFont(const QFont&)`
- `setFontFilters(QFontComboBox::FontFilters)`

- `setWritingSystem(QFontDatabase::WritingSystem)`

C++ - Transkript font seçici

```cpp
auto* fontSec = new QFontComboBox(this);
fontSec->setCurrentFont(QFont("DejaVu Sans"));
fontSec->setFontFilters(QFontComboBox::ScalableFonts);

connect(fontSec, &QFontComboBox::currentFontChanged,
        this, [](const QFont& f){
    qDebug() << "Yeni font:" << f.family();
});
```

Alıştırma

1. `fontFilters` hangi türde değerleri alabilir? En az 3 tane bayrak (flag) yaz.

Cevabın: _____

2. `writingSystem` ne işe yarar? Türkçe metinler için hangi değer en uygun olur?

Cevabın: _____

Cevap Anahtarı

1. `AllFonts`, `ScalableFonts`, `NonScalableFonts`, `MonospacedFonts`, `ProportionalFonts`. Bu bayraklar listede hangi font tiplerinin görüneceğini sınırlar; örneğin kod editöründe `MonospacedFonts` mantıklıdır.
2. Combo'da yalnızca belirli bir yazı sistemini destekleyen fontları gösterir. Türkçe karakterler Latin alfabesinin genişletilmiş kümesinde olduğundan `QFontDatabase::Latin` uygundur; bu seçim Cyrillic-only veya CJK-only fontları listeden eler.

Kategori 03 Bitiş Quizi

Seçim ve Tercih – Mini Quiz

1. (Boşluk doldurma) QCheckBox'ın üç durumlu (Unchecked / PartiallyChecked / Checked) olmasını sağlayan property _____ dir.

Cevabın: _____

2. (Kavramsal) İki radyo butonu farklı QGroupBox'lara yerleştirilirse aynı anda ikisi de seçili kalabilir mi? Cevabını *neden* ile birlikte yaz.

Cevabın: _____

3. (Eşleştirme) Aşağıdaki sinyalleri taşıdıkları parametre tipi ile eşleştir:

- `QComboBox::currentIndexChanged` → _____
- `QCheckBox::toggled` → _____
- `QButtonGroup::idClicked` → _____

4. (Kod çıktısı) Aşağıdaki kodda `qDebug()` ne yazdırır?

C++ - Soru 4

```
QComboBox cb;
cb.addItems({"MAT101", "FIZ102", "KIM103"});
cb.setCurrentText("FIZ102");
qDebug() << cb.currentIndex();
```

Cevabın: _____

5. (Mikro kod) 3 radyo butonlu (Sec A / Sec B / Sec C) bir QButtonGroup `*grup` olduğunu varsay. Aktif olarak seçili butonun metnini `QString aktif` değişkenine atayan 1-2 satırlık kodu yaz.

Kategori 03 Quiz – Cevap Anahtarı

1. `tristate` (`setTristate(true)` ile aktif edilir).
2. Hayır, her ikisi de seçili kalabilir; yani biri diğerini *etkilemez*. Çünkü `autoExclusive` özelliği yalnızca *aynı parent widget* içindeki radyolar arasında çalışır. Farklı QGroupBox'lar farklı parent oldukları için her grup kendi içinde bağımsız davranır.
3. Eşleştirme:

- `currentIndexChanged` → `int` (yeni indeks)
- `toggled` → `bool` (yeni seçili durumu)
- `idClicked` → `int` (tıklanan butonun ID'si)

4. `1` yazdırır. `addItems` sırayla 0, 1, 2 indekslerini doldurur; `setCurrentText("FIZ102")` listede mevcut bir öğeyi seçtiği için ilgili indeks olan `1` geçerli olur.
5. İki satırlık olası çözüm:

C++ - Cevap 5

```
auto* btn = grup->checkedButton();
QString aktif = btn ? btn->text() : QString{};
```

Tek satır isteniyorsa: `QString aktif = grup->checkedButton() ? grup->checkedButton()->text() : "";`

4 Konteyner ve Düzen

Widget'ları gruplayan ve yerleştiren yapılar. İki alt-grup var:

- **Konteyner widget'lar:** QGroupBox, QScrollArea, QSplitter, QStackedWidget, QTabWidget, QToolBox – çocuklarını görsel olarak çerçeveler.
- **Layout yöneticileri:** QHBoxLayout, QVBoxLayout, QGridLayout, QFormLayout, QStackedLayout – çocukların konumunu yönetir.

Bolum 5'te (*(bkz. kitap)*) layout'lara giriş yapmıştık; burada tüm seçenekleri toparlıyoruz. Ayrıca widget boyutlandırma mantığının kalbi olan QSizePolicy de bu kategoride ele alınır.

4.1 QGroupBox

QObject → QWidget → QGroupBox

Başlıklı, çerçeveli bir gruplama kutusudur. İçine bir layout yerleştirilir ve o layout'a çocuk widget'lar eklenir. OBYS form ekranında "Öğrenci Bilgileri", "İletişim", "Notlar" gibi mantıksal blokları görsel olarak ayırmak için idealdir. İsteğe bağlı olarak *checkable* (işaretlenebilir) yapılabilir; bu durumda başlığın yanına bir kutucuk gelir ve grup kapatıldığında içindeki tüm çocuklar otomatik olarak devre dışı kalır.

Önemli Properties

Property	Tip	Açıklama
title	QString	Çerçevenin üstündeki başlık
checkable	bool	Başlık yanında kutucuk göster
checked	bool	checkable ise grup açık mı?
alignment	Qt::Alignment	Başlık hizalaması (sol/orta/sağ)
flat	bool	Yalnızca üst çizgi (minimal stil)

Signals

- `toggled(bool on)` – checkable grup açıldı/kapandı
- `clicked(bool checked)` – başlık kutucuğuna tıklandı

Slots

- `setChecked(bool)`, `setTitle(QString)`

C++ - Öğrenci bilgileri grubu

```
auto* grup = new QGroupBox("Ogrenci Bilgileri", this);
auto* form = new QFormLayout(grup);
form->addRow("Ad:",      new QLineEdit(grup));
form->addRow("Numara:",  new QLineEdit(grup));
form->addRow("Bolum:",   new QComboBox(grup));
```

Alıştırma

1. `checkable` özelliği `true` olan bir QGroupBox'ın işlevsel davranışı nedir? Bir senaryo örneği ver.

Cevabın: _____

2. `flat` property'si açıldığında görsel olarak ne değişir? Hangi durumlarda tercih edilir?

Cevabın: _____

Cevap Anahtarı

1. Başlığın yanına bir kutucuk eklenir. Kutucuk işaretli değilken grup içindeki tüm çocuk widget'lar otomatik olarak devre dışı (disabled) hale gelir. Tipik kullanım: "Otomatik yedekleme" grubu – kullanıcı kutucuğu açmadıkça frekans, hedef klasör vb. ayarlar gri görünür.
2. `flat` açıkken çerçevenin yan ve alt kenarları çizilmez; sadece üstte ince bir çizgi ve başlık kalır. Modern, sadeleştirilmiş arayüzlerde veya zaten başka bir çerçeve içinde yer alan iç içe gruplarda görsel kirliliği azaltmak için kullanılır.

4.2 QScrollArea

QObject → QWidget → QFrame → QAbstractScrollArea → QScrollArea

İçeriği pencereye sığamayacak kadar büyük olduğunda otomatik olarak yatay/dikey kaydırma çubukları gösteren konteynerdir. `setWidget()` çağrısı ile *tek bir* çocuk widget alır; genellikle bu çocuk büyük bir form, uzun bir liste veya geniş bir resim olur. OBYS uzun anket formlarında veya çok sayıda not satırını gösteren bir panelde sıkça kullanılır.

Önemli Properties

Property	Tip	Açıklama
widget	QWidget*	İçindeki tek widget
widgetResizable	bool	Çocuğu otomatik yeniden boyutla
horizontalScrollBarPolicy	Qt::ScrollBarPolicy	AsNeeded / AlwaysOff / AlwaysOn
verticalScrollBarPolicy	Qt::ScrollBarPolicy	AsNeeded / AlwaysOff / AlwaysOn
alignment	Qt::Alignment	İçeriğin görünüm bölgesindeki hizası

Slots

- `setWidget(QWidget*)`, `ensureWidgetVisible(QWidget*)`
- `ensureVisible(int x, int y)`

C++ - Uzun form için scroll area

```cpp
auto* icerik = new QWidget;
auto* form = new QFormLayout(icerik);
for (int i = 1; i <= 30; ++i)
    form->addRow(QString("Soru %1:").arg(i), new QLineEdit);

auto* scroll = new QScrollArea(this);
scroll->setWidget(icerik);
scroll->setWidgetResizable(true);
```

Alıştırma

1. `widgetResizable true` ile `false` arasındaki davranış farkı nedir?

Cevabın: _____

2. QScrollArea doğrudan birden fazla widget tutabilir mi? Birden fazla bileşen göstermek istesek nasıl yaparız?

Cevabın: _____

Cevap Anahtarı

1. `widgetResizable=true` iken iç widget, scroll area'nın görünüm bölgesinin (viewport) genişliğine göre yeniden boyutlandırılır; layout'lar düzgün çalışır. `false` iken iç widget kendi `sizeHint()` değerinde sabit kalır ve gerekirse küçük görünür ya da scroll yapılmaz.
2. Hayır, yalnızca tek bir widget tutabilir. Birden fazla bileşen göstermek için bir QWidget oluşturup üzerine bir layout (QVBoxLayout vb.) kurarız ve tüm bileşenleri o layout'a ekleriz; sonra bu sarmalayıcı widget'ı `setWidget()` ile scroll area'ya veririz.

4.3 QSplitter

QObject → QWidget → QFrame → QSplitter

Çocuk widget'larını yatay veya dikey olarak yerleştirir ve aralarına kullanıcının fareyle sürükleyerek yeniden boyutlandırabileceği bir tutamak (handle) koyar. IDE'lerdeki kod editörü ile dosya ağacı arasındaki bölme veya OBYS ana penceresinde sol taraftaki not listesi ile sağ taraftaki grafik paneli arasındaki bölme tipik kullanımıdır.

Önemli Properties

Property	Tip	Açıklama
orientation	Qt::Orientation	Horizontal / Vertical
handleWidth	int	Tutamak kalınlığı (pixel)
childrenCollapsible	bool	Çocuklar tamamen kapatılabilir mi
opaqueResize	bool	Sürükleme sırasında canlı önizleme

Signals

- `splitterMoved(int pos, int index)` – tutamak hareket etti

Slots / Yöntemler

- `addWidget(QWidget*)`, `insertWidget(int, QWidget*)`
- `setSizes(QList<int>)`, `sizes()`
- `saveState()`, `restoreState(QByteArray)`

C++ - Yatay splitter: Notlar / Grafikler

```
auto* splitter = new QSplitter(Qt::Horizontal, this);
splitter->addWidget(new NotListesiPanel(splitter));
splitter->addWidget(new GrafikPanel(splitter));
splitter->setSizes({200, 500});
splitter->setChildrenCollapsible(false);
```

Alıştırma

1. `saveState()` ve `restoreState()` hangi amaçla kullanılır? Tipik bir senaryo açıkla.

Cevabın: _____

2. `childrenCollapsible false` olduğunda kullanıcı arayüzünde ne değişir?

Cevabın: _____

Cevap Anahtarı

1. Splitter'ın bölme oranlarını bir `QByteArray` olarak diske (örn. QSettings ile) kaydedip uygulama tekrar açıldığında geri yüklemek için kullanılır. Böylece kullanıcının ayarladığı pencere düzeni oturumlar arası korunur.
2. `false` iken kullanıcı bir paneli sürükleyerek sıfır genişliğe (yani tamamen gizli) indiremez; her panel en az kendi minimum boyutunda kalmak zorundadır. Bu, panellerin yanlışlıkla kaybolmasını engeller.

4.4 QStackedWidget

QObject → QWidget → QFrame → QStackedWidget

İçine eklenen birden fazla widget'tan yalnızca birini, yani *aktif sayfayı* görünür yapar. Diğerleri bellektedir ama ekranda gözükmez. Sihirbaz (wizard) adımları, ayar kategorileri arasında geçiş veya OBYS giriş ekranı → ana panel geçişi gibi senaryolar için idealdir. Sekme barı göstermez; aktif sayfayı kod ile değiştirirsin.

Önemli Properties

Property	Tip	Açıklama
currentIndex	int	Aktif sayfanın indeksi
currentWidget	QWidget*	Aktif sayfanın widget'ı
count	int	Toplam sayfa sayısı (read-only)

Signals

- `currentChanged(int index)` – aktif sayfa değişti
- `widgetRemoved(int index)` – bir sayfa kaldırıldı

Slots

- `addWidget(QWidget*)`, `insertWidget(int, QWidget*)`
- `setCurrentIndex(int)`, `setCurrentWidget(QWidget*)`

C++ - 3 sayfalı wizard

```cpp
auto* stack = new QStackedWidget(this);
stack->addWidget(new HosgeldinSayfasi);    // index 0
stack->addWidget(new AyarlarSayfasi);      // index 1
stack->addWidget(new BitisSayfasi);        // index 2

connect(ileriBtn, &QPushButton::clicked, this, [stack]{
    stack->setCurrentIndex(stack->currentIndex() + 1);
});
```

Alıştırma

1. QStackedWidget ile QTabWidget arasındaki en temel görsel/işlevsel fark nedir?

Cevabın: _____

2. Aktif sayfayı değiştirmenin iki farklı yolunu yaz.

Cevabın: _____

Cevap Anahtarı

1. QTabWidget kullanıcının doğrudan tıklayabileceği sekme barı sağlar; QStackedWidget ise hiçbir görsel kontrol sunmaz. Sayfayı program kodu (örn. "İleri" butonu) değiştirir. Bu yüzden wizard, login akışı veya navigasyonun başka bir mekanizmayla yapıldığı senaryolar için uygundur.
2. İndeks ile: `stack->setCurrentIndex(2);`
Widget pointer ile: `stack->setCurrentWidget(ayarlarSayfasi);`

4.5 QTabWidget

QObject → QWidget → QTabWidget

İçerdiği her widget'ı kendi başlığı olan bir *sekme* olarak gösterir. Kullanıcı sekme barından tıklayarak sayfalar arası geçer. OBYS ana penceresinde "Notlar", "Grafikler", "Trend", "Ayarlar" gibi ana bölümleri ayırmak için en sık tercih edilen konteynerdir.

Önemli Properties

Property	Tip	Açıklama
currentIndex	int	Aktif sekme
tabPosition	QTabWidget::TabPosition	North / South / West / East
tabsClosable	bool	Her sekmede "X" butonu göster
movable	bool	Sekmeler sürüklenip sıralanabilir
documentMode	bool	Sade, doküman tarzı çerçeve

Signals

- `currentChanged(int index)` – sekme değişti
- `tabCloseRequested(int index)` – "X" tıklandı
- `tabBarClicked(int index)` – sekme bar'a tıklandı

Slots

- `addTab(QWidget*, QString)`, `insertTab(int, QWidget*, QString)`
- `removeTab(int)`, `setCurrentIndex(int)`

C++ - OBYS ana sekmeleri

```cpp
auto* tabs = new QTabWidget(this);
tabs->addTab(new NotPanel,     "Notlar");
tabs->addTab(new GrafikPanel, "Grafikler");
tabs->addTab(new TrendPanel,  "Trend");
tabs->setTabsClosable(true);

connect(tabs, &QTabWidget::tabCloseRequested,
        tabs, &QTabWidget::removeTab);
```

Alıştırma

1. `tabsClosable true` yaparsak "X" butonuna tıklandığında sekme *otomatik olarak* silinir mi? Açıkla.

Cevabın: _____

2. `documentMode` açıldığında görsel olarak ne değişir? Hangi tür uygulamalarda tercih edilir?

Cevabın: _____

Cevap Anahtarı

1. Hayır, otomatik silmez. `tabsClosable=true` yalnızca "X" butonunu görsel olarak ekler ve tıklandığında `tabCloseRequested(int)` sinyalini yayınlar. Sekmenin gerçekten kaldırılması için bu sinyali `removeTab(int)` slot'una bağlamak (veya "Kaydedilmemiş değişiklik var mı?" gibi bir kontrol yapıp ondan sonra silmek) gerekir.
2. `documentMode=true` iken sekme barı etrafındaki çerçeve incelir veya kaldırılır; sekmeler doğrudan içeriğe yapışmış gibi görünür. Web tarayıcıları, kod editörleri, çoklu-doküman uygulamaları gibi her sekmenin bir "belge" olduğu durumlarda tercih edilir.

4.6 QToolBox

QObject → QWidget → QFrame → QToolBox

Akordiyon (accordion) tarzı bir konteynerdir. Her sayfanın başlık çubuğu vardır; kullanıcı bir başlığa tıkladığında o sayfa açılır, diğerleri kapanır. Çizim/grafik araç paletleri veya sınırlı yatay alana sahip kenar çubukları için tipik seçimdir.

Önemli Properties

Property	Tip	Açıklama
currentIndex	int	Aktif (açık) sayfa indeksi
count	int	Toplam sayfa sayısı (read-only)

Signals

- `currentChanged(int index)` – aktif sayfa değişti

Slots / Yöntemler

- `addItem(QWidget*, QString text)`
- `setCurrentIndex(int)`, `setCurrentWidget(QWidget*)`
- `setItemEnabled(int, bool)`, `setItemText(int, QString)`

C++ - Çizim aracı paleti

```
auto* arac = new QToolBox(this);
arac->addItem(new CizimAraclariPanel,  "Cizim");
arac->addItem(new RenkPaletiPanel,     "Renk");
arac->addItem(new BoyutAyariPanel,     "Boyut");
arac->setCurrentIndex(0);
```

Alıştırma

1. QTabWidget yerine QToolBox kullanmayı hangi durumlarda tercih edersin?

Cevabın: _____

2. QToolBox sayfalarının tamamı aynı boyda olmak zorunda mıdır?

Cevabın: _____

Cevap Anahtarı

1. Yatay alan sınırlı (örn. dar bir kenar çubuğu) ve sayfalar arasında dikey olarak gezinmenin daha doğal olduğu durumlarda; ayrıca grafik tasarım programlarındaki araç paletleri gibi her sayfanın kendi içinde küçük bir bölüm olduğu yerlerde tercih edilir.
2. Hayır. QToolBox her sayfa için ayrı bir başlık satırı gösterir ve aktif sayfa kendi içeriği kadar yer kaplar. Sayfalar farklı boylarda olabilir; QToolBox toplam boyutuna göre kendini ayarlar.

4.7 QHBoxLayout

QObject → QLayout → QBoxLayout → QHBoxLayout

Çocuk widget'ları yatay olarak (soldan sağa) sıralayan layout yöneticisidir. Layout sistemi Qt'nin esnek arayüz oluşturma mekanizmasının kalbidir; pencere yeniden boyutlandırıldığında çocukların boyutları otomatik ayarlanır. Buton kutuları (OK / İptal), araç çubukları, durum çubuğu içerikleri için tipik seçimdir.

Önemli Properties

Property	Tip	Açıklama
spacing	int	Çocuklar arası piksel boşluğu
contentsMargins	QMargins	Layout kenar boşlukları
alignment	Qt::Alignment	İçeriğin hizalanması

Slots / Yöntemler

- `addWidget(QWidget*, int stretch=0, Qt::Alignment=)`
- `addLayout(QLayout*, int stretch=0)`
- `addStretch(int stretch=0)`, `addSpacing(int)`
- `insertWidget(int, QWidget*)`

C++ - Sağa yaslı OK / İptal butonları

```
auto* butonSatiri = new QHBoxLayout;
butonSatiri->addStretch();                // Sol esnek bosluk
butonSatiri->addWidget(new QPushButton("Tamam"));
butonSatiri->addWidget(new QPushButton("Iptal"));
butonSatiri->setContentsMargins(0, 8, 0, 0);
```

Alıştırma

1. `addStretch()` fonksiyonu ne işe yarar? Layout içinde nasıl davranır?

Cevabın: _____

2. `setContentsMargins(left, top, right, bottom)` hangi boşlukları kontrol eder? `spacing` ile farkı nedir?

Cevabın: _____

Cevap Anahtarı

1. Layout'a "esneyen boş alan" ekler. Pencere boyutlandırıldığında bu boş alan elastik şekilde büyür/küçülür ve diğer widget'ları sabit boyutlarında bırakır. Tipik kullanım: butonları sağa yaslamak için solda bir `addStretch()` koymak; ortalamak için her iki yana koymak.
2. `contentsMargins` layout'un dış kenarları ile içerideki çocuklar arasındaki boşluğu (yani layout'un kendi iç kenar payını) kontrol eder. `spacing` ise çocuk widget'ların *birbirleri arasındaki* boşluğu belirler. Biri çerçeve, diğeri elemanlar arası mesafe.

4.8 QVBoxLayout

QObject → QLayout → QBoxLayout → QVBoxLayout

Çocuk widget'ları dikey olarak (üstten alta) sıralayan layout yöneticisidir. Qt programlamada *en sık kullanılan* layout sınıfıdır: bir form ekranı, ana pencere içeriği, kenar çubuğu veya diyaloglar genellikle bir QVBoxLayout ile başlar. QHBoxLayout ile aynı temel sınıftan (QBoxLayout) türer ve aynı API'yi kullanır; tek fark yöndür.

Önemli Properties

Property	Tip	Açıklama
spacing	int	Çocuklar arası dikey boşluk
contentsMargins	QMargins	Layout'un kendi iç kenar payı

Slots / Yöntemler

- `addWidget(QWidget*, int stretch=0, Qt::Alignment=)`
- `addLayout(QLayout*, int stretch=0)`
- `addStretch(int stretch=0)`

C++ - Dikey form düzeni

```
auto* ana = new QVBoxLayout(this);
ana->addWidget(new QLabel("<h2>Ogrenci Ekle</h2>"));

auto* form = new QFormLayout;
form->addRow("Ad:",      new QLineEdit);
form->addRow("Numara:", new QLineEdit);
ana->addLayout(form);

auto* btnSatiri = new QHBoxLayout;
btnSatiri->addStretch();
btnSatiri->addWidget(new QPushButton("Kaydet"));
ana->addLayout(btnSatiri);
```

Alıştırma

1. `addWidget(widget, stretch)` fonksiyonundaki `stretch` parametresi ne anlama gelir?

Cevabın: _____

2. QVBoxLayout ile QBoxLayout arasındaki fark nedir?

Cevabın: _____

Cevap Anahtarı

1. Stretch çarpanı, fazladan kullanılabilir alanın çocuklar arasında nasıl paylaştırılacağını belirler. Örneğin iki widget'tan birine `stretch=1` diğerine `stretch=2` verirsen, ekstra alan 1:2 oranında dağıtılır. Varsayılan `0` olduğu için widget'lar sadece kendi `sizeHint` boyutlarında kalır.
2. QBoxLayout genel temel sınıftır ve yapıcısında `Qt::Orientation` parametresi alır (`LeftToRight`, `TopToBottom` vb.). QVBoxLayout ise "yön = TopToBottom" olarak hazır gelen kolaylık alt sınıfıdır. QHBoxLayout aynı şekilde "LeftToRight" kısayolu sağlar.

4.9 QGridLayout

QObject → QLayout → QGridLayout

Çocuk widget'ları satır/sütun ızgarasına yerleştiren güçlü bir layout yöneticisidir. Bir hücre tek bir widget tutabileceği gibi, bir widget birden fazla satır/sütun da kaplayabilir (spanning). Hesap makinesi tuş takımı, giriş formları, karmaşık ayar ekranları için tipik tercihtir.

Önemli Properties

Property	Tip	Açıklama
horizontalSpacing	int	Sütunlar arası boşluk
verticalSpacing	int	Satırlar arası boşluk
contentsMargins	QMargins	Layout'un kenar payı

Slots / Yöntemler

- `addWidget(widget, row, col)`
- `addWidget(widget, row, col, rowSpan, colSpan)`
- `setColumnStretch(int col, int stretch)`
- `setRowStretch(int row, int stretch)`
- `setColumnMinimumWidth(int col, int w)`

C++ - Giriş ekranı

```cpp
auto* g = new QGridLayout(this);
g->addWidget(new QLabel("Kullanici:"), 0, 0);
g->addWidget(new QLineEdit,            0, 1);
g->addWidget(new QLabel("Sifre:"),     1, 0);
g->addWidget(new QLineEdit,            1, 1);
auto* btn = new QPushButton("Giris");
g->addWidget(btn, 2, 0, 1, 2);   // 2 sutun kaplar
g->setColumnStretch(1, 1);
```

Alıştırma

1. `addWidget(widget, row, col, rowSpan, colSpan)` çağrısındaki `rowSpan` ve `colSpan` ne anlama gelir? Bir senaryo örneği ver.

Cevabın: _____

2. `setColumnStretch(1, 1)` çağrısı bir 2 sütunlu grid'de ne sağlar?

Cevabın: _____

Cevap Anahtarı

1. `rowSpan` widget'ın kaç satır, `colSpan` ise kaç sütun kaplayacağını belirtir. Örnek: 2 sütunlu bir formun en altına yerleştirilen "Giriş" butonu hem 0. hem 1. sütunu kaplasın diye `(row=2, col=0, rowSpan=1, colSpan=2)` verilir. Tablonun başlığı için tüm sütunlara yayılan bir QLabel da aynı mantıkla yapılır.
2. 0. sütun varsayılan stretch (0) olduğu için sadece kendi içeriği kadar yer tutar; 1. sütuna stretch 1 verildiği için pencere genişledikçe tüm fazladan alan 1. sütuna gider. Sonuç: etiket sütunu sabit, giriş alanı sütunu esnek olur.

4.10 QFormLayout

QObject → QLayout → QFormLayout

İki sütunlu, "etiket-alan" (label-field) çiftleri için optimize edilmiş bir layout'tur. Sol sütunda açıklayıcı etiketler, sağ sütunda giriş alanları yer alır. OBYS not giriş formu, öğrenci ekleme diyaloğu, ayar ekranları için ideal çünkü hem hizalama hem platform kılavuzlarına uygun davranış otomatik gelir.

Önemli Properties

Property	Tip	Açıklama
labelAlignment	Qt::Alignment	Etiket hizalaması
formAlignment	Qt::Alignment	Tüm formun hizalaması
fieldGrowthPolicy	QFormLayout::FieldGrowthPolicy	Alanların büyüme şekli
rowWrapPolicy	QFormLayout::RowWrapPolicy	Etiket alt satıra geçer mi

Slots / Yöntemler

- `addRow(QString label, QWidget* field)`
- `addRow(QWidget* label, QWidget* field)`
- `insertRow(int, ...)`, `removeRow(int)`

C++ - Öğrenci ekleme formu

```
auto* f = new QFormLayout(this);
f->addRow("Ad:",      new QLineEdit);
f->addRow("Numara:",  new QLineEdit);
f->addRow("Bolum:",   new QComboBox);
f->addRow("Not:",     new QSpinBox);
f->setLabelAlignment(Qt::AlignRight);
```

Alıştırma

1. `addRow(QString, QWidget*)` ile `addRow(QLabel*, QWidget*)` arasındaki fark nedir? Hangisini ne zaman tercih edersin?

Cevabın: _____

2. Aynı formu QGridLayout ile yazsaydık ne tür ekstra iş yapmamız gerekirdi?

Cevabın: _____

Cevap Anahtarı

1. İlk form QString alır ve dahili olarak otomatik bir QLabel oluşturur. İkinci form ise hazır bir QLabel pointer'ı alır; bu sayede etiketin font, renk, alignment veya "buddy" (`setBuddy` ile klavye kısayolu) gibi özelliklerini ince ayarlayabilirsin. Basit metin için ilkini, özel görünüm/davranış için ikincisini kullanırsın.
2. QGridLayout ile her satır için manuel olarak `(row, 0)` hücresine etiket, `(row, 1)` hücresine alanı eklemen, sütun stretch'lerini ayarlaman ve etiket hizalamasını kendin yönetmen gerekirdi. Ayrıca platforma göre (macOS sağa hizalı, Windows sola) etiket düzenini elle yapman gerekir; QFormLayout bunu otomatik halleder.

4.11 QStackedLayout

QObject → QLayout → QStackedLayout

QStackedWidget'in layout karşılığıdır: aynı alanı paylaşan birden fazla alt widget'tan birini görünür yapar. Aradaki fark şudur: QStackedWidget bir *widget*'tır (kendisi ekrana yerleşir), QStackedLayout ise bir *layout*'tur (başka bir widget'ın içine kurulur). Mevcut bir konteyner widget'a ekstra wrapper eklemeden "sayfa değiştirme" özelliği vermek isteyince tercih edilir.

Önemli Properties

Property	Tip	Açıklama
currentIndex	int	Aktif sayfa
currentWidget	QWidget*	Aktif sayfa widget'ı
stackingMode	QStackedLayout::StackingMode	StackOne / StackAll

Signals

- `currentChanged(int index)`
- `widgetRemoved(int index)`

Slots / Yöntemler

- `addWidget(QWidget*)`, `insertWidget(int, QWidget*)`
- `setCurrentIndex(int)`, `setCurrentWidget(QWidget*)`

C++ - Wizard adımları

```
auto* stack = new QStackedLayout(this);
stack->addWidget(new HosgeldinSayfasi);
stack->addWidget(new BilgiSayfasi);
stack->addWidget(new BitisSayfasi);
stack->setCurrentIndex(0);
```

Alıştırma

1. QStackedWidget ile QStackedLayout aynı işlevi yapıyorsa hangi durumda hangisini tercih edersin?

Cevabın: _____

2. `stackingMode` property'sinin `StackOne` ve `StackAll` değerleri arasındaki fark nedir?

Cevabın: _____

Cevap Anahtarı

1. QStackedWidget hazır bir widget olduğu için başka bir layout'a doğrudan eklenebilir; sayfa değiştirme işlemini "bağımsız bir bileşen" olarak modellemek istiyorsan uygun. QStackedLayout ise mevcut bir widget'ın içine kurulur; o widget'a ek bir wrapper eklemeden çoklu sayfa yönetimi vermek istediğinde, özellikle de form gibi başka mantıksal layout'larla iç içe çalışırken tercih edilir.
2. `StackOne` (varsayılan): yalnızca aktif sayfa görünür, diğerleri gizlenir. `StackAll`: tüm sayfalar görünür kalır ama yalnızca aktif olan girdi alır; bu mod yarı saydam katmanlar veya overlay senaryolarında işe yarar.

4.12 QSizePolicy

(QSizePolicy bir widget değil, bir değer sınıfıdır)

QSizePolicy bir widget değildir; her QWidget'ın `sizePolicy` özelliği üzerinden taşıdığı bir *yapılandırma nesnesidir*. Layout sistemine "bu widget yatay/dikey eksende nasıl davranmalı; yer artarsa büyüsün mü, sıkışırsa küçülsün mü?" bilgisini taşır. Layout yöneticilerinin doğru davranabilmesi için kritik bir kavramdır; arayüzünde widget'lar beklemediğin yerlere yerleşiyorsa büyük ihtimalle suçlu QSizePolicy'dir.

Politika Değerleri (Qt::Policy)

Değer	Davranış
Fixed	sizeHint sabit; büyümez/küçülmez
Minimum	sizeHint minimum, büyüyebilir
Maximum	sizeHint maksimum, küçülebilir
Preferred	sizeHint tercih, gerekirse esner
Expanding	Mümkün olduğunca büyür
MinimumExpanding	sizeHint min; ek alanı paylaşır
Ignored	sizeHint dikkate alınmaz

Tipik Yöntemler

- `QWidget::setSizePolicy(QSizePolicy::Policy h, QSizePolicy::Policy v)`
- `QSizePolicy::setHorizontalStretch(int)`
- `QSizePolicy::setVerticalStretch(int)`

C++ - Yatayda esneyen buton

```
auto* btn = new QPushButton("Genis Buton", this);
btn->setSizePolicy(QSizePolicy::Expanding,
                   QSizePolicy::Fixed);
// Yatayda mevcut alani doldurur, dikeyde sabit kalir.
```

Alıştırma

1. `Preferred` ile `Expanding` arasındaki davranış farkını bir layout senaryosuyla açıkla.

Cevabın: _____

2. Bir widget'a `Fixed` politikası vermek hangi durumlarda mantıklıdır? Bir örnek ver.

Cevabın: _____

Cevap Anahtarı

1. İki widget yan yana, biri `Preferred` biri `Expanding` olsun. Pencere genişlediğinde fazladan alanın neredeyse tamamı `Expanding` widget'a gider; `Preferred` widget kendi `sizeHint` boyutunda kalır. İki widget da `Preferred` olsa fazladan alan eşit paylaşılır. Yani `Expanding` "yer kapma önceliği yüksek" demektir.
2. İçeriği boyutu önceden belli ve değişmemesi gereken widget'lar için: bir simge butonu (32x32 ikon), sabit boyutlu küçük bir önizleme alanı, hesap makinesi tuşları, sabit boyutlu bir logo QLabel'ı. `Fixed` verirsek layout asla bu widget'ı büyütmez veya küçültmez.

Kategori Bitiş Quizi

5 Soruluk Karma Quiz

1. Bir QHBoxLayout içinde butonları sağa yaslamak için hangi method çağrılır ve nereye eklenir?

---

2. (Eşleştirme) Konteyneri uygun amaca eşle:

QTabWidget	?
QStackedWidget	?
QSplitter	?
QGroupBox	?

3. (Kod okuma) Aşağıdaki kodda iki widget eklenmiş bir QHBoxLayout var. Pencere geniş açıldığında `w1` ve `w2` boyutları yaklaşık hangi oranda olur?

C++

```
auto* h = new QHBoxLayout(this);
h->addWidget(w1, 1);
h->addWidget(w2, 2);
```

---

4. QFormLayout'un, aynı işi yapan QGridLayout'a göre en belirgin avantajı nedir?

---

5. `QSizePolicy::Expanding` ile `QSizePolicy::MinimumExpanding` arasındaki fark nedir?

Cevap Anahtarı

1. `addStretch()` çağrılır ve butonlardan *önce* (yani layout'un başına) eklenir. Esneyen boş alan butonları sağa iter. Ortalamak için her iki yana da `addStretch()` eklenir.
2. QTabWidget → kullanıcının sekme barından doğrudan tıklayarak gezindiği çoklu sayfa görünümü; QStackedWidget → aynı alanda kod ile değiştirilen sayfalar (wizard, login akışı); QSplitter → kullanıcının sürükleyerek yeniden boyutlandırabileceği iki/üç bölgeli düzen; QGroupBox → ilgili widget'ları başlık ve çerçeveyle görsel olarak gruplama.
3. Stretch çarpanları `1` ve `2`, dolayısıyla fazladan alan 1:2 oranında paylaşılır; `w1` pencerenin yaklaşık 1/3'ünü, `w2` ise yaklaşık 2/3'ünü kaplar (çocukların minimum sizeHint'leri saygı görmek koşuluyla).
4. Etiket-alan çiftlerinin hizalamasını, iki sütunlu düzeni ve platform kılavuzlarına uygun davranışı (ör. macOS'ta sağa hizalı etiket) otomatik halleder; QGridLayout ile bunların hepsini elle ayarlamak gerekir. Ayrıca `addRow(QString, QWidget*)` ile kod kısa ve okunaklı olur.
5. `Expanding`: widget'ın `sizeHint` değeri sadece bir öneridir; layout fazladan tüm alanı bu widget'a vermeye eğilimlidir ve gerekirse `sizeHint`'in altına da inebilir. `MinimumExpanding`: `sizeHint` aynı zamanda bir alt sınırdır – widget bunun altına inmez ama fazladan alan verildiğinde yine büyür. Yani ikincisi "minimum garantili, gerisi esnek" demektir.

5 Görüntüleme Widget'ları

Veriyi göstermek veya zaman tabanlı seçim yaptırmak için kullanılan widget'lar. İlerleme (QProgressBar), sayısal gösterge (QLCDNumber), takvim (QCalendarWidget) ve tarih/saat seçimi (QDateEdit, QTimeEdit, QDateTimeEdit) bu kategoride yer alır. OBYS raporlama (*(bkz. kitap)*) akışında ilerleme göstergesi kritiktir.

5.1 QProgressBar

QObject → QWidget → QProgressBar

Bir işlemin ilerleme durumunu yatay veya dikey bir çubukla gösterir. 0–100 arası klasik yüzde gösterimi yapabildiği gibi, `minimum == maximum == 0` ayarlandığında "meşgul" (belirsiz) modda sürekli akış animasyonu çalıştırır. OBYS toplu rapor üretirken yüzlerce öğrencinin not ortalaması hesaplanırken kullanıcıya bu çubukla geri bildirim verilir.

Önemli Properties

Property	Tip	Açıklama
value	int	Şu anki ilerleme değeri
minimum	int	Aralığın alt sınırı
maximum	int	Aralığın üst sınırı
format	QString	Görüntülenecek metin (%p, %v, %m)
textVisible	bool	Yüzde/metnin görünüp görünmediği
orientation	enum	Horizontal / Vertical
invertedAppearance	bool	Sağdan sola dolar
textDirection	enum	Dikey modda yazı yönü

Signals

- `valueChanged(int value)` – value her değiştiğinde yayınlanır

Slots

- `setValue(int)` – İlerleme değerini ayarlar
- `setRange(int min, int max)` – Aralığı ayarlar
- `setFormat(const QString&)` – Metin biçimini ayarlar
- `reset()` – Çubuğu başa alır

C++ - Mini ornek: Mesgul (belirsiz) mod

```
auto* bar = new QProgressBar(this);
bar->setRange(0, 0);                // belirsiz/mesgul mod
bar->setTextVisible(false);
// Uzun surec tamamlandiginda:
// bar->setRange(0, 100);
// bar->setValue(100);
```

Alıştırma

1. `textVisible` property'si `false` yapıldığında çubuk nasıl görünür ve hangi durumda bunu tercih edersin?

Cevabın: _____

2. Bir QProgressBar'da "42 / 100" biçiminde metin göstermek istiyorsun. `setFormat` çağrısını nasıl yazarsın?

Cevabın: _____

Cevap Anahtarı

1. Çubuğun üzerindeki yüzde/metin yazısı kaybolur, sadece dolan renkli kısım görünür. Belirsiz (meşgul) modda veya çubuğun yanına ayrı bir QLabel ile özel metin yerleştirdiğinde tercih edilir.
2. `bar->setFormat("%v / %m");` – Burada `%v` şu anki değeri (value), `%m` maksimumu temsil eder. `%p` ise yüzde değerini verir.

5.2 QLCDNumber

QObject → QWidget → QFrame → QLCDNumber

Sayısal değerleri klasik yedi-segment LCD görünümünde gösteren widget. Saat, geri sayım, puan tablosu, retro tarz dashboard'lar veya laboratuvar arayüzleri için idealdir. OBYS'de bir öğretmen panelinde sınıf ortalamasını büyük ve okunaklı göstermek için kullanılabilir. Ondalıklı sayıları, tam sayıları ve dize içinden sayısal okumayı destekler.

Önemli Properties

Property	Tip	Açıklama
digitCount	int	Görüntülenecek hane sayısı
value	double	Şu anki ondalık değer
intValue	int	Şu anki tam sayı değer
mode	enum	Hex / Dec / Oct / Bin
segmentStyle	enum	Outline / Filled / Flat
smallDecimalPoint	bool	Ondalık nokta küçük gösterilsin mi

Signals

- `overflow()` – Görüntülenecek değer hane sayısına sığmadığında yayınlanır

Slots

- `display(int)` – Tam sayı gösterir
- `display(double)` – Ondalık sayı gösterir
- `display(const QString&)` – Dize olarak gösterir
- `setDigitCount(int)` – Hane sayısını ayarlar
- `setMode(Mode)` – Sayı tabanını ayarlar

C++ - Mini ornek: OBYS sinif ortalamasi paneli

```
auto* lcd = new QLCDNumber(5, this); // 5 hane (orn. 87.50)
lcd->setSegmentStyle(QLCDNumber::Filled);
lcd->setSmallDecimalPoint(true);
double ortalama = 87.5;
lcd->display(ortalama);
```

Alıştırma

1. `mode` property'si `QLCDNumber::Hex` yapıldığında `lcd->display(255)` ne gösterir? Ya `Bin` yapılırsa?

Cevabın: _____

2. `overflow()` sinyali tam olarak ne zaman yayınlanır? 2 haneli bir LCD'de `display(1234)` çağrılırsa ne olur?

Cevabın: _____

Cevap Anahtarı

1. Hex modda `255` değeri `FF` olarak görünür. Bin modda ise `11111111` (8 hane). Bu nedenle `Bin` modu kullanırken `digitCount` değerinin yeterince büyük olduğundan emin olmak gerekir.
2. Görüntülenecek değer mevcut `digitCount` ile sığmadığında `overflow()` sinyali yayınlanır ve LCD bir hata göstergesi (genelde tüm tireler) gösterir. 2 haneli LCD'de `display(1234)` bu sinyali tetikler.

5.3 QCalendarWidget

QObject → QWidget → QCalendarWidget

Tam aylık takvim görünümü sunan etkileşimli widget. Kullanıcı ay/yıl arasında gezinerek bir tarih seçebilir; özel günler biçimlendirilebilir (örneğin tatil günleri kırmızı). OBYS'de dönem başlangıç tarihi seçici, sınav günü planlayıcı veya etkinlik takvimi olarak kullanılabilir. Pop-up değil, tam görünür bir takvimdir.

Önemli Properties

Property	Tip	Açıklama
selectedDate	QDate	Seçili tarih
minimumDate	QDate	Seçilebilir en küçük tarih
maximumDate	QDate	Seçilebilir en büyük tarih
firstDayOfWeek	enum	Haftanın ilk günü (Pazartesi vb.)
gridVisible	bool	Hücre çizgileri görünür mü
selectionMode	enum	SingleSelection / NoSelection

Signals

- `selectionChanged()` – Seçili tarih değişti
- `clicked(const QDate&)` – Bir tarihe tıklandı
- `activated(const QDate&)` – Tarih çift tıklandı
- `currentPageChanged(int year, int month)` – Sayfa değişti

Slots

- `setSelectedDate(const QDate&)` – Seçili tarihi ayarla
- `showToday()` – Bugüne git
- `showPreviousMonth()` / `showNextMonth()` – Ay gezinmesi
- `setDateTextFormat(const QDate&, const QTextCharFormat&)` – Özel biçim

C++ - Mini ornek: Donem baslangic secimi

```
auto* takvim = new QCalendarWidget(this);
takvim->setFirstDayOfWeek(Qt::Monday);
takvim->setMinimumDate(QDate(2026, 1, 1));
takvim->setMaximumDate(QDate(2026, 12, 31));
connect(takvim, &QCalendarWidget::clicked,
        this, [](const QDate& d){
    qDebug() << "Donem baslangici:" << d;
});
```

Alıştırma

1. Takvimin ilk gününü Pazartesi yapmak için hangi çağrı yapılır? (Tam C++ satırını yaz.)

Cevabın: _____

2. `selectionMode` property'si hangi iki değeri alabilir ve aralarındaki fark nedir?

Cevabın: _____

Cevap Anahtarı

1. `takvim->setFirstDayOfWeek(Qt::Monday);` – Bu çağrı haftanın ilk sütununu Pazartesi yapar (varsayılan locale'e göre Pazar olabilir).
2. İki değer vardır: `QCalendarWidget::SingleSelection` (varsayılan, kullanıcı tek bir tarih seçebilir) ve `QCalendarWidget::NoSelection` (kullanıcı tarih seçemez, takvim sadece görüntüleme amaçlıdır). Çoklu tarih seçimi yoktur.

5.4 QDateEdit

QObject → QWidget → QAbstractSpinBox → QDateTimeEdit → QDateEdit

Tarih girişi için kompakt bir spin kutusu. Tek satırlık alanda gün/ay/yıl bölümleri ok tuşları veya klavye ile değiştirilebilir. `calendarPopup` açıkken sağ tarafta bir ok belirir ve tıklandığında küçük bir takvim açılır. OBYS form ekranlarında "Doğum Tarihi" veya "Kayıt Tarihi" alanları için en uygun seçenektir; QCalendarWidget'a göre çok daha az yer kaplar.

Önemli Properties

Property	Tip	Açıklama
date	QDate	Şu anki tarih değeri
minimumDate	QDate	Alt sınır
maximumDate	QDate	Üst sınır
displayFormat	QString	Görüntülenme biçimi (örn. "dd.MM.yyyy")
calendarPopup	bool	Pop-up takvim açılsın mı

Signals

- `dateChanged(const QDate&)` – Tarih değişti
- `dateTimeChanged(const QDateTime&)` – Üst sınıftan miras

Slots

- `setDate(const QDate&)` – Tarihi ayarla
- `setDisplayFormat(const QString&)` – Biçimi ayarla
- `setDateRange(const QDate& min, const QDate& max)` – Aralık

C++ - Mini ornek: Ogrenci dogum tarihi alani

```
auto* dogum = new QDateEdit(this);
dogum->setDisplayFormat("dd.MM.yyyy");
dogum->setCalendarPopup(true);
dogum->setMinimumDate(QDate(1950, 1, 1));
dogum->setMaximumDate(QDate::currentDate());
dogum->setDate(QDate(2005, 9, 15));
```

Alıştırma

1. `calendarPopup` property'si `true` yapıldığında widget'ın görünümü ve davranışı nasıl değişir?

Cevabın: _____

2. `displayFormat = "dddd, d MMMM yyyy"` ayarlandığında `QDate(2026, 4, 11)` nasıl görünür?

Cevabın: _____

Cevap Anahtarı

1. Widget'ın sağında küçük bir aşağı ok düğmesi belirir; kullanıcı bu düğmeye tıklayınca açılan pop-up'ta tam bir QCalendarWidget görünür ve fareyle tarih seçilebilir. Klavye ile de bölüm bölüm düzenleme hâlâ mümkündür.
2. "Cumartesi, 11 Nisan 2026" (locale'e bağlı) görünür. `dddd` = haftanın günü adı, `d` = gün numarası, `MMMM` = ay adı, `yyyy` = 4 haneli yıl. Diğer önemli kodlar: `dd` (2 haneli gün), `MM` (2 haneli ay), `MMM` (kısa ay adı), `yy` (2 haneli yıl).

5.5 QTimeEdit

QObject → QWidget → QAbstractSpinBox → QDateTimeEdit → QTimeEdit

Saat girişi için kompakt spin kutusu. QDateEdit ile simetrik bir yapıya sahiptir; sadece tarih yerine saat-dakika-saniye bölümleri vardır. OBYS'de ders saati başlangıç-bitiş alanları, sınav süreleri veya yoklama saati için kullanılır. Klavye ve ok tuşlarıyla bölüm bölüm düzenleme yapılabilir.

Önemli Properties

Property	Tip	Açıklama
time	QTime	Şu anki saat değeri
minimumTime	QTime	Alt sınır
maximumTime	QTime	Üst sınır
displayFormat	QString	Görüntülenme biçimi (örn. "hh:mm:ss")

Signals

- `timeChanged(const QTime&)` – Saat değeri değişti
- `dateTimeChanged(const QDateTime&)` – Üst sınıftan miras

Slots

- `setTime(const QTime&)` – Saati ayarla
- `setDisplayFormat(const QString&)` – Biçimi ayarla
- `setTimeRange(const QTime& min, const QTime& max)` – Aralık

C++ - Mini ornek: Ders saati baslangici

```
auto* dersBaslangic = new QTimeEdit(this);
dersBaslangic->setDisplayFormat("hh:mm");
dersBaslangic->setMinimumTime(QTime(8, 0));
dersBaslangic->setMaximumTime(QTime(20, 0));
dersBaslangic->setTime(QTime(9, 30));
connect(dersBaslangic, &QTimeEdit::timeChanged,
        this, [](const QTime& t){
    qDebug() << "Yeni saat:" << t.toString("hh:mm");
});
```

Alıştırma

1. 12-saat formatında AM/PM göstermek için `displayFormat` nasıl yazılır? Tam dize ile cevap ver.

Cevabın: _____

2. `displayFormat = "hh:mm AP"` ayarlandığında `QTime(14, 30)` nasıl görünür? Ya `"HH:mm"` ile?

Cevabın: _____

Cevap Anahtarı

1. `edit->setDisplayFormat("hh:mm AP");` – Burada `AP` büyük harfli AM/PM eki ekler (`ap` ise küçük harfli am/pm ekler). `hh` bu durumda 1–12 aralığında saat gösterir.
2. `"hh:mm AP"` ile `QTime(14, 30)` değeri “02:30 PM” olarak görünür. `"HH:mm"` ile (büyük H = 24-saat) “14:30” olarak görünür. Küçük `h` 12 saat formatında, büyük `H` 24 saat formatındadır.

5.6 QDateTimeEdit

QObject → QWidget → QAbstractSpinBox → QDateTimeEdit

Tarih ve saati tek bir spin kutusunda birleştiren widget. QDateEdit ve QTimeEdit’in ortak ata sınıfıdır; tek başına da kullanılabilir. `calendarPopup` açıkken sağ tarafta takvim açılır, saat bölümleri ise klavyeyle düzenlenir. OBYS’de sınav tarihi+saati seçimi, randevu planlama veya dosya zaman damgası gibi alanlar için en doğru widget’tır.

Önemli Properties

Property	Tip	Açıklama
dateTime	QDateTime	Şu anki tarih+saat
displayFormat	QString	Görüntülenme biçimi
calendarPopup	bool	Pop-up takvim açılsın mı
currentSection	enum	Klavye kursörünün olduğu bölüm
minimumDateTime	QDateTime	Alt sınır
maximumDateTime	QDateTime	Üst sınır

Signals

- `dateTimeChanged(const QDateTime&)` – Tarih+saat değişti
- `dateChanged(const QDate&)` – Sadece tarih bölümü değişti
- `timeChanged(const QTime&)` – Sadece saat bölümü değişti

Slots

- `setDateTime(const QDateTime&)` – Değer ayarla
- `setDateRange(const QDate&, const QDate&)` – Tarih aralığı
- `setMinimumDateTime(const QDateTime&)` – Alt sınır
- `setMaximumDateTime(const QDateTime&)` – Üst sınır

C++ - Mini ornek: Sinav tarihi + saati

```cpp
auto* sinav = new QDateTimeEdit(this);
sinav->setDisplayFormat("dd.MM.yyyy hh:mm");
sinav->setCalendarPopup(true);
sinav->setMinimumDateTime(QDateTime::currentDateTime());
sinav->setDateTime(QDateTime(QDate(2026, 6, 15), QTime(10, 0)));
connect(sinav, &QDateTimeEdit::dateTimeChanged,
        this, [](const QDateTime& dt){
    qDebug() << "Sinav:" << dt.toString("dd.MM.yyyy hh:mm");
});
```

Alıştırma

1. `currentSection` property'si tam olarak ne bilgisi verir? Hangi durumda işine yarar?

Cevabın: _____

2. `setDateTimeRange` ile `setDateRange` + `setTimeRange` kullanmak arasındaki fark nedir? Hangi durumda hangisi tercih edilir?

Cevabın: _____

Cevap Anahtarı

1. `currentSection` klavye kursörünün şu anda hangi bölümün üzerinde olduğunu döner: `DaySection`, `MonthSection`, `YearSection`, `HourSection`, `MinuteSection`, `SecondSection`, `AmPmSection` vb. Kullanıcı kısayol tuşlarına bastığında doğru bölümü artırmak/azaltmak veya özelleştirilmiş davranış vermek için işine yarar.
2. `setDateTimeRange(min, max)` tek bir QDateTime aralığı tanımlar; örneğin "2026-04-11 09:00'tan 2026-04-11 17:00'a kadar" gibi sürekli bir aralık. `setDateRange` + `setTimeRange` ise bağımsız iki sınır koyar: "2026 Nisan ayında, sadece 09:00–17:00 arası saatler". İlki tek bir zaman penceresi, ikincisi günlük tekrarlayan saat aralığı içindir.

Kategori Bitiş Quizi

5 Soruluk Karma Quiz

1. Bir QProgressBar'ı "sonsuz" (belirsiz/meşgul) moda almak için `minimum` ve `maximum` hangi değerlere ayarlanır?

---

2. QCalendarWidget'ın seçili tarihini almak için çağrılan metodun imzası nedir ve hangi tür döndürür?

---

3. (Kod okuma) Aşağıdaki QLCDNumber'da hangi değer görünür ve ek olarak hangi sinyal yayınlanır?

C++

```
QLCDNumber lcd;
lcd.setDigitCount(4);
lcd.display(12345);
```

---

4. Bir QDateTimeEdit'in `displayFormat` property'si `"yyyy.MM.dd hh:mm"` olarak ayarlandığında, ekranda "2026/04/11 14:30" biçimi görünür mü? Neden?

---

5. (Mikro kod) Bir QProgressBar'ı 0–200 aralığında, ilerlemeyi yüzde olarak gösterecek şekilde ayarlayan üç satırlık kod parçasını yaz:

Cevap Anahtarı

1. `minimum == maximum == 0` olmalıdır: `bar->setRange(0, 0);`. Bu durumda Qt çubuğu sürekli akan bir animasyona alır ve gerçek bir yüzde göstermez.
2. `QDate QCalendarWidget::selectedDate() const` metodu çağrılır ve bir QDate nesnesi döndürür. Örnek: `QDate d = takvim->selectedDate();`.
3. Görünen değer "12345" tam olarak 5 hane gerektirdiği için 4 haneye sığmaz; LCD bir taşma göstergesi (genelde tireler) gösterir ve `overflow()` sinyali yayınlanır.
4. Hayır, "2026.04.11 14:30" görünür. Çünkü `displayFormat` string'inde noktalı ayraç (`.`) tanımlandı. Ekrana çıkacak biçim bu format string'iyle bire bir eşleşir; eğik çizgi göstermek istenseydi format `"yyyy/MM/dd hh:mm"` olmalıydı.
5. `auto* bar = new QProgressBar(this);`
`bar->setRange(0, 200);`
`bar->setFormat("%p%");`
(`%p` yüzde değerini, `%%` ise tek bir yüzde işaretini basar. `format` varsayılanı zaten `"%p%"`'dir, ama açıkça yazmak güvenlidir.)

6 Item View Widget'ları

Liste, tablo ve ağaç görünümleri. Qt'nin Model/View mimarisinin (*(bkz. kitap)*) iki farklı yüzü var:

- **Convenience widget'lar** – QListWidget, QTableWidget, QTreeWidget – kendi içlerinde model'i gizler, hızlı prototipleme için.
- **View + Model ikilisi** – QListView, QTableView, QTreeView – ayrı bir model'e bağlanır. Büyük veri veya özel işlem için.

OBYS'de QTableView + QAbstractTableModel çiftini (*(bkz. kitap)*) ders notlarını göstermek için kullanmıştık. Bu kategori her iki yaklaşımı da özetler.

6.1 QListWidget

QObject → QWidget → QFrame → QAbstractScrollArea → QAbstractItemView → QListView → QListWidget

Basit metin veya ikon listesi gösteren convenience widget. Model'i kendi içinde saklar; `QListWidgetItem` nesneleri ekleyip çıkararak listeyi yönetirsin. Küçük listelere idealdir (son açılan dosyalar, yapılacaklar listesi, OBYS'de seçili dersler paneli gibi). Performans kritik veya çok büyük veri varsa QListView + harici model tercih edilmeli.

Önemli Properties

Property	Tip	Açıklama
count	int	Toplam öğe sayısı (salt-okunur)
currentRow	int	Seçili satır indeksi
currentItem	QListWidgetItem*	Seçili öğe işaretçisi
selectionMode	enum	Single/Multi/Extended seçim
sortingEnabled	bool	Otomatik alfabetik sıralama

Signals

- `currentRowChanged(int)`
- `itemClicked(QListWidgetItem*)`
- `itemDoubleClicked(QListWidgetItem*)`
- `itemChanged(QListWidgetItem*)` – öğe metni değişince

Slots

- `addItem`, `addItems`, `insertItem`
- `clear`, `setCurrentRow`, `sortItems`

C++ - Son açılan dosyalar listesi

```
auto* sonDosyalar = new QListWidget(this);
sonDosyalar->addItem("ogrenciler.csv");
sonDosyalar->addItem("notlar_2026_bahar.csv");
sonDosyalar->addItem("mufredat.json");
sonDosyalar->setSelectionMode(QAbstractItemView::SingleSelection);

connect(sonDosyalar, &QListWidget::itemDoubleClicked,
```

```
        this, [](QListWidgetItem* item){
    qDebug() << "Aciliyor:" << item->text();
});
```

Alıştırma

1. `QListWidgetItem` ile QListWidget arasındaki ilişki nedir? Bir öğenin metnini sonradan değiştirmek için hangi metot kullanılır?

Cevabın: _____

2. `sortingEnabled true` yapıldığında ne olur? Liste değiştirildikten sonra sıralama otomatik mi yenilenir, yoksa `sortItems()` çağırmak mı gerekir?

Cevabın: _____

Cevap Anahtarı

1. QListWidget bir kapsayıcıdır; QListWidgetItem tek tek öğelerdir. Her öğe metin, ikon, font, veri (`Qt::UserRole`) gibi bilgileri tutar. Bir öğenin metnini değiştirmek için `item->setText("yeni metin")` kullanılır; bu çağrı `itemChanged` sinyalini tetikler.
2. `sortingEnabled = true` olduğunda yeni eklenen her öğe sıralı pozisyona otomatik yerleştirilir. `sortItems()` elle çağırmaya gerek kalmaz; ancak sıralama yönünü (`Qt::AscendingOrder` / `DescendingOrder`) değiştirmek istersen yine `sortItems(order)` çağırırsın.

6.2 QTableWidget

QObject → QWidget → QFrame → QAbstractScrollArea → QAbstractItemView → QTableView → QTableWidget

Hücre bazlı, hızlı kurulan tablo. Kendi iç model'i ile gelir, sen sadece `QTableWidgetItem` nesnelerini hücrelere yerleştirirsin. Özel bir model yazmak istemediğinde – örneğin küçük bir not giriş formu, ayar tablosu veya rapor önizleme – en pratik seçimdir. Veriyi başka kaynaklardan çekiyor veya filtrelemek istiyorsan QTableView + QAbstractTableModel daha uygun.

Önemli Properties

Property	Tip	Açıklama
rowCount	int	Satır sayısı
columnCount	int	Sütun sayısı
showGrid	bool	Hücre kılavuz çizgileri
sortingEnabled	bool	Başlığa tıklayınca sırala
alternatingRowColors	bool	Çift/tek satır renk değişimi

Signals

- `cellClicked(int row, int col)`
- `cellDoubleClicked(int row, int col)`
- `cellChanged(int row, int col)` – içerik düzenlenince
- `itemChanged(QTableWidgetItem*)`

Slots

- `setItem`, `setRowCount`, `setColumnCount`
- `setHorizontalHeaderLabels`, `setVerticalHeaderLabels`
- `clear`, `insertRow`, `removeRow`

C++ - Hızlı not giriş tablosu

```cpp
auto* tablo = new QTableWidget(10, 3, this);
tablo->setHorizontalHeaderLabels({"Ogrenci No", "Vize", "Final"});

for (int satir = 0; satir < 10; ++satir) {
    auto* numara = new QTableWidgetItem(QString::number(20240001 +
        satir));
    numara->setFlags(numara->flags() & ~Qt::ItemIsEditable); //
        salt-okunur
    tablo->setItem(satir, 0, numara);
    tablo->setItem(satir, 1, new QTableWidgetItem("0"));
    tablo->setItem(satir, 2, new QTableWidgetItem("0"));
}
```

Alıştırma

1. `QTableWidgetItem::setFlags()` hangi amaçla kullanılır? Bir hücrenin salt-okunur olmasını sağlayan flag kombinasyonu nedir?

Cevabın: _____

2. `cellChanged` sinyali ile `itemChanged` sinyali arasındaki fark nedir? Hangisini kullanmak daha güvenlidir?

Cevabın: _____

Cevap Anahtarı

1. `setFlags()` bir hücrenin hangi etkileşimlere açık olduğunu belirler (seçilebilir, düzenlenebilir, sürüklenebilir, işaretlenebilir vb.). Salt-okunur yapmak için `Qt::ItemIsEditable` bayrağını kaldırmak yeterli: `item->setFlags(item->flags() & ~Qt::ItemIsEditable);` Genellikle `Qt::ItemIsSelectable | Qt::ItemIsEnabled` bırakılır.
2. `cellChanged(row, col)` sadece satır/sütun indeksini verir; hızlı erişim için pratik. `itemChanged(item)` ise değişen `QTableWidgetItem*` işaretçisini verir, item'a özel veriye (UserRole, font, vb.) doğrudan erişebilirsin. İkisi de aynı anda tetiklenir; ihtiyacına göre seç.

6.3 QTreeWidget

QObject → QWidget → QFrame → QAbstractScrollArea → QAbstractItemView → QTreeView → QTreeWidget

Hiyerarşik (ağaç) yapıları gösteren convenience widget. Bölüm → Ünite → Konu, klasör → alt klasör, kategori → ürün gibi katmanlı verileri göstermek için idealdir. Her düğüm bir `QTreeWidgetItem`'dır ve `addChild()` ile alt düğüm alabilir. OBYS müfredat ağacı veya ders programı gezgini için tipik örnektir.

Önemli Properties

Property	Tip	Açıklama
columnCount	int	Sütun sayısı
headerLabels	QStringList	Başlık etiketleri
topLevelItemCount	int	Kök düğüm sayısı (salt-okunur)
rootIsDecorated	bool	Kök düğümlerde aç/kapa oku

Signals

- `itemClicked(QTreeWidgetItem*, int column)`
- `itemDoubleClicked(QTreeWidgetItem*, int)`
- `itemExpanded(QTreeWidgetItem*)`
- `itemCollapsed(QTreeWidgetItem*)`

Slots

- `addTopLevelItem`, `insertTopLevelItem`
- `expandAll`, `collapseAll`
- `clear`

C++ - OBYS müfredat ağacı

```cpp
auto* agac = new QTreeWidget(this);
agac->setHeaderLabels({"Mufredat"});

auto* bolum6 = new QTreeWidgetItem(agac, QStringList{"Bolum 6: Qt GUI"});
auto* unite1 = new QTreeWidgetItem(bolum6, QStringList{"Unite 1: MVC"});
new QTreeWidgetItem(unite1, QStringList{"Konu 1: Model/View"});
new QTreeWidgetItem(unite1, QStringList{"Konu 2: Roller"});

auto* unite2 = new QTreeWidgetItem(bolum6, QStringList{"Unite 2: Model"});
new QTreeWidgetItem(unite2, QStringList{"Konu 1: QAbstractTableModel"});

agac->expandAll();
```

Alıştırma

1. `addChild()` ile `addTopLevelItem()` arasındaki fark nedir? Aynı item iki ayrı parent'a eklenebilir mi?

Cevabın: _____

2. `rootIsDecorated` property'si ne gösterir? `false` yapılırsa kullanıcı kök düğümleri nasıl açıp kapatabilir?

Cevabın: _____

Cevap Anahtarı

1. `addTopLevelItem()` öğeyi ağacın köküne ekler (parent'ı ağacın kendisidir). `addChild()` ise mevcut bir `QTreeWidgetItem`'ın altına yerleştirir. Bir item'ın tek bir parent'ı olabilir; iki yere eklemek istersen önce `clone()` ile kopyasını almalısın.
2. `rootIsDecorated`, kök seviyedeki düğümlerin yanında aç/kapa oku (disclosure triangle) gösterir. `false` yapılırsa kök düğümlerde ok kaybolur; kullanıcı çift tıklayarak veya klavye ok tuşları (sağ/sol) ile yine açıp kapatabilir, ya da programatik olarak `expandItem()` çağırırsın.

6.4 QListView

QObject → QWidget → QFrame → QAbstractScrollArea → QAbstractItemView → QListView

Harici bir model'e bağlanan düşük seviye liste görünümü. Convenience kuzeni QListWidget'tan farklı olarak veriyi kendi içinde tutmaz; `setModel()` ile bir QAbstractItemModel türevi (örneğin QStringListModel, QStandardItemModel veya kendi modelin) bağlanır. Büyük veri, dinamik kaynak (veritabanı, ağ) veya proxy filtresi gerektiğinde tercih edilir. Ayrıca `IconMode` ile ızgara/grid görünümü sunabilir.

Önemli Properties

Property	Tip	Açıklama
model	QAbstractItemModel*	Bağlı veri modeli
selectionModel	QItemSelectionModel*	Seçim yöneticisi
viewMode	enum	ListMode / IconMode
spacing	int	Öğeler arası boşluk
gridSize	QSize	IconMode'da hücre boyutu

Signals

- `clicked(const QModelIndex&)`
- `doubleClicked(const QModelIndex&)`
- `activated(const QModelIndex&)`

Slots

- `setModel`, `setSelectionModel`
- `setRootIndex`, `scrollTo`

C++ - QStringListModel ile basit liste

```cpp
auto* model = new QStringListModel(this);
model->setStringList({"Matematik I", "Fizik I",
                      "Programlamaya Giris", "Lineer Cebir"});

auto* liste = new QListView(this);
liste->setModel(model);
liste->setSelectionMode(QAbstractItemView::SingleSelection);

connect(liste, &QListView::clicked, this,
        [model](const QModelIndex& idx){
    qDebug() << "Secilen ders:" << model->data(idx, Qt::DisplayRole);
});
```

Alıştırma

1. QListView, QListWidget'a hangi durumlarda tercih edilir? En az iki somut senaryo yaz.

Cevabın: _____

2. `viewMode = QListView::IconMode` ne işe yarar? Bu modda hangi property'ler önem kazanır?

Cevabın: _____

Cevap Anahtarı

1. (a) Veri zaten harici bir kaynakta – veritabanı, JSON, ağ – tutuluyorsa kendi model'ini yazıp QListView'e bağlamak veriyi iki yerde tutmaktan kurtarır. (b) QSortFilterProxyModel ile filtreleme/sıralama uygulanacaksa proxy ancak harici model üzerine kurulabilir. (c) Çok sayıda öğe (on binler) varken QListWidget kopya nesneler tuttuğu için bellek ve hız açısından dezavantajlıdır.
2. `IconMode`, öğeleri ikon + altyazı olarak ızgara şeklinde gösterir (dosya gezgini ikon görünümü gibi). Bu modda `gridSize`, `spacing`, `flow`, `movement` ve `wrapping` property'leri görsel yerleşimi belirler.

6.5 QTableView

QObject → QWidget → QFrame → QAbstractScrollArea → QAbstractItemView → QTableView

Model/View mimarisinde tablonun görsel tarafı. QAbstractTableModel (veya QAbstractItemModel) türevi bir modele bağlanır. OBYS'de ders notlarını `NotlarModel : QAbstractTableModel` sınıfından türetip QTableView'e bağlamıştık (*(bkz. kitap)*). Sıralama, filtreleme (QSortFilterProxyModel) ve binlerce satırı verimle göstermek için tasarlanmıştır.

Önemli Properties

Property	Tip	Açıklama
showGrid	bool	Hücre kılavuz çizgileri
sortingEnabled	bool	Başlığa tıklayınca sırala
alternatingRowColors	bool	Çift/tek satır renkleri
horizontalHeader	QHeaderView*	Sütun başlığı widget'ı
verticalHeader	QHeaderView*	Satır başlığı widget'ı

Signals

- `clicked(const QModelIndex&)`
- `doubleClicked(const QModelIndex&)`
- `activated(const QModelIndex&)`

Slots

- `setModel`, `resizeColumnsToContents`
- `resizeRowsToContents`, `selectRow`, `selectColumn`
- `sortByColumn`

C++ - OBYS NotlarModel'e bağlı tablo

```cpp
auto* model = new NotlarModel(this); // QAbstractTableModel turevi
model->yukle("notlar_2026_bahar.csv");

auto* tablo = new QTableView(this);
tablo->setModel(model);
tablo->setSortingEnabled(true);
tablo->setAlternatingRowColors(true);
tablo->resizeColumnsToContents();

// Filtreleme icin proxy
auto* proxy = new QSortFilterProxyModel(this);
proxy->setSourceModel(model);
tablo->setModel(proxy);
```

Alıştırma

1. OBYS notlarını göstermek için QTableWidget yerine QTableView + `NotlarModel` kullanmanın hangi avantajları vardır? En az iki avantaj yaz.

Cevabın: _____

2. `alternatingRowColors` property'si neye yarar? Hangi stil renkleri kullanır (palette rolü)?

Cevabın: _____

Cevap Anahtarı

1. (a) Veri tek noktada tutulur; iş mantığı (kaydet, yükle, hesapla) modelin içindedir, view sadece görünüm sağlar. (b) Aynı modeli birden fazla view'e bağlayabilirsin (tablo + grafik). (c) QSortFilterProxyModel ile filtreleme/sıralama tek satırda eklenir. (d) Binlerce satırda QTableWidget'tan çok daha hızlı, çünkü yalnızca görünen hücreler için `data()` çağrılır.
2. `alternatingRowColors = true` olduğunda satırlar sırayla iki farklı renkte boyanır; uzun listelerde okunabilirliği artırır. Renkler `QPalette::Base` ve `QPalette::AlternateBase` rollerinden alınır; tema değişince otomatik uyum sağlar.

6.6 QTreeView

QObject → QWidget → QFrame → QAbstractScrollArea → QAbstractItemView → QTreeView

Hiyerarşik model'in görsel tarafı. Kendisine bağlanan model QAbstractItemModel türevi olmalı (parent/child ilişkisini desteklediği için QAbstractTableModel yetmez). En sık karşılaştığın örnek QFileSystemModel: dosya sistemi gezginlerinde, ayar ağaçlarında, JSON/XML görüntüleyicilerde QTreeView kullanılır. Sütun başına farklı veri (boyut, tarih, tip) gösterebilir.

Önemli Properties

Property	Tip	Açıklama
rootIsDecorated	bool	Kök düğümlerde aç/kapa oku
indentation	int	Seviye başına piksel girinti
animated	bool	Açma/kapama animasyonu
expandsOnDoubleClick	bool	Çift tıkla aç/kapa
uniformRowHeights	bool	Tüm satırlar aynı yükseklikte

Signals

- `expanded(const QModelIndex&)`
- `collapsed(const QModelIndex&)`
- `clicked(const QModelIndex&)`
- `activated(const QModelIndex&)`

Slots

- `setModel`, `setRootIndex`
- `expand`, `collapse`, `expandAll`, `collapseAll`

C++ - QFileSystemModel ile dosya gezgini

```
auto* model = new QFileSystemModel(this);
model->setRootPath(QDir::homePath());

auto* gezgin = new QTreeView(this);
gezgin->setModel(model);
gezgin->setRootIndex(model->index(QDir::homePath()));
gezgin->setUniformRowHeights(true); // performans icin
gezgin->setColumnWidth(0, 250);

connect(gezgin, &QTreeView::doubleClicked, this,
        [model](const QModelIndex& idx){
    qDebug() << "Acildi:" << model->filePath(idx);
});
```

Alıştırma

1. QFileSystemModel ile çalışırken QTreeView hangi dizini kök olarak gösterir? Sadece ev dizinini göstermek istersek hangi iki çağrı yapılmalıdır?

Cevabın: _____

2. `uniformRowHeights = true` ne sağlar? Performans açısından neden önemlidir?

Cevabın: _____

Cevap Anahtarı

1. QFileSystemModel varsayılan olarak tüm dosya sistemini modelleyebilir; QTreeView ise modelin kökünü gösterir. Sadece ev dizinini göstermek için modele `model->setRootPath(QDir::homePath())` ile başlangıç dizinini söyleriz, sonra view'a `gezgin->setRootIndex(model->index(QDir::homePath()))` diyerek görünür kökü o dizine sabitleriz.
2. `uniformRowHeights = true` olduğunda Qt tüm satırların aynı yükseklikte olduğunu varsayar ve tek bir satırın yüksekliğini ölçüp gerisini hesaplar. Aksi halde her satır için `sizeHint()` çağrılır; binlerce satırda bu çok pahalıdır. Açılınca scroll ve çizim hissedilir derecede hızlanır; ancak satırların gerçekten farklı yüksekliklere ihtiyacı varsa görsel bozulma olur.

6.7 Model Mimarisi — Kısa Hatırlatma

Bölüm 6'da öğrendiğimiz Model/View mimarisini kısaca özetleyelim (*(bkz. kitap)*). Qt'de model sınıfları üç seviyededir:

- QAbstractItemModel – en genel taban; ağaç, tablo, liste her şeyi modelleyebilir. QTreeView bunu bekler.
- QAbstractTableModel – iki boyutlu tablo modeli; parent/child yok. OBYS NotlarModel için kullandığımız taban.
- QStandardItemModel – hazır, item tabanlı genel amaçlı model. Hızlı prototipleme için.

Modeldeki her hücrenin birden çok *rolü* olabilir:

- `Qt::DisplayRole` – ekranda gösterilecek metin
- `Qt::EditRole` – düzenleme kutusunda görünecek değer
- `Qt::DecorationRole` – ikon/renk

- `Qt::ToolTipRole` – fare üstüne gelince çıkan ipucu
- `Qt::UserRole` – kendi özel verin (id, nesne, vb.)

Filtreleme veya sıralama için QSortFilterProxyModel kullanılır: kaynak modelin önüne koyulan bir *ara katman*dır, view onu gerçek model gibi görür.

C++ - QAbstractTableModel iskeleti

```cpp
class NotlarModel : public QAbstractTableModel {
    Q_OBJECT
    QVector<Not> m_notlar;
public:
    int rowCount(const QModelIndex&) const override {
        return m_notlar.size();
    }
    int columnCount(const QModelIndex&) const override { return 3; }

    QVariant data(const QModelIndex& idx, int role) const override {
        if (role != Qt::DisplayRole) return {};
        const auto& n = m_notlar[idx.row()];
        switch (idx.column()) {
            case 0: return n.ogrenciNo;
            case 1: return n.dersKodu;
            case 2: return n.puan;
        }
        return {};
    }
    QVariant headerData(int section, Qt::Orientation o, int role) const
        override;
};
```

Alıştırma

1. `Qt::DisplayRole` ile `Qt::EditRole` arasındaki fark nedir? Bir tarih hücresi için ikisi neden farklı dönmelidir?

Cevabın: _____

2. QSortFilterProxyModel'in `setFilterRegularExpression()` metodu ne yapar? Hangi sütun üzerinde çalışacağı nasıl belirlenir?

Cevabın: _____

Cevap Anahtarı

1. `DisplayRole` kullanıcıya gösterilen biçimlenmiş metni döner (örn. "11 Nisan 2026"); `EditRole` ise düzenleme sırasında editor'a verilecek ham değeri döner (örn. `QDate(2026,4,11)`). Aynı veriyi iki farklı amaçla sunmamızı sağlar: gözlem için okunabilir, düzenleme için makineye uygun.
2. `setFilterRegularExpression(QRegularExpression)`, proxy'den geçen satırları belirtilen düzenli ifadeye göre süzer: ifadeyle eşleşmeyen satırlar gizlenir. Hangi sütunda eşleşme aranacağı `setFilterKeyColumn(int)` ile belirlenir; -1 verilirse tüm sütunlar kontrol edilir.

Kategori 06 Bitiş Quizi

Item View Quizi

1. (Eşleştirme) Aşağıdaki senaryolar için en uygun view'i seç: QListWidget, QTableView, QTreeView.

- 5 öğelik son-açılan-dosya listesi → _____
- 50.000 öğrencilik not tablosu (filtreli) → _____
- Dosya sistemi gezgini → _____

2. `QListWidget::item(i)` çağrısı ile `QListView::model()->index(i, 0)` çağrısı kavramsal olarak hangi açıdan farklıdır?

Cevabın: _____

3. (Kod okuma) Aşağıdaki kod çalıştığında ekrana hangi öğeler görünür? Tablonun (liste görünümünün) ilk satırındaki metin nedir?

C++

```
QStringListModel model(QStringList{"A","B","C"});
QListView view;
view.setModel(&model);
view.show();
```

Cevabın: _____

4. QSortFilterProxyModel bağlı olduğu kaynak model değiştiğinde otomatik olarak güncellenir mi? Açıkla.

Cevabın: _____

5. (Mikro kod) Bir QTableView'de sütun genişliklerinin hücre içeriklerine otomatik uyum sağlaması için gereken *tek satır* kodu yaz.

Cevabın: _____

Cevap Anahtarı — Kategori 06 Quizi

1. (a) 5 öğelik son-dosya listesi → QListWidget (küçük, statik, hızlı kurulum).
(b) 50.000 öğrencilik not tablosu → QTableView (QAbstractTableModel + QSortFilterProxyModel ile filtre).
(c) Dosya sistemi gezgini → QTreeView (QFileSystemModel ile birlikte; hiyerarşik yapı).
2. `QListWidget::item(i)` doğrudan bir `QListWidgetItem*` döner — yani convenience widget'ın kendi tuttuğu somut nesneye erişirsin. `model()->index(i, 0)` ise bir QModelIndex döner; bu yalnızca modeldeki bir konuma *başvuru*dur, kendisi veri tutmaz. Veriyi okumak için `model->data(idx, role)` çağırman gerekir. İlki nesne tabanlı, ikincisi indeks/rol tabanlıdır.
3. Liste görünümünde üç satır görünür: "A", "B", "C". İlk satırdaki metin **A**'dır. QStringListModel, verilen string listesini doğrudan `DisplayRole` olarak sunar.
4. Evet, otomatik güncellenir. Proxy, kaynak modelin `dataChanged`, `rowsInserted`, `rowsRemoved`, `modelReset` gibi sinyallerine bağlıdır; kaynakta her değişiklikte filtre/sıralama mantığını yeniden uygular ve view'a ileterek güncelleme yapar.
5. `tablo->resizeColumnsToContents();`

7 Pencere Elemanları

Ana uygulama penceresinin kurucu parçaları: QMainWindow iskelet, QMenuBar + QMenu menüleri, QToolBar araç çubuğu, QStatusBar alt bildirim çubuğu, QDockWidget yüzen paneller, QSystemTrayIcon sistem tepsisi, QWizard adım adım sihirbaz ve QMdiArea çoklu belge arayüzü. OBYS'nin ana penceresi (*(bkz. kitap)*) bu parçaların çoğundan yararlanır.

7.1 QMainWindow

QObject → QWidget → QMainWindow

Standart bir masaüstü uygulamasının iskeletidir. İçinde merkez widget (central widget), üst menü çubuğu, araç çubukları, dock panelleri ve alt durum çubuğu için hazır yuvalar bulunur. OBYS ana penceresi (*(bkz. kitap)*) doğrudan bu sınıftan türetilir; menü, toolbar ve durum çubuğu otomatik olarak doğru konumlara yerleşir. Pencere durumunu (toolbar konumu, dock'ların açık/kapalı olması) `saveState` / `restoreState` ile diske yazıp geri yükleyebilirsin.

Önemli Properties

Property	Tip	Açıklama
centralWidget	QWidget*	Pencerenin orta alanı
menuBar	QMenuBar*	Üst menü çubuğu
statusBar	QStatusBar*	Alt durum çubuğu
unifiedTitleAndToolBarOnMac	bool	macOS birleşik başlık
documentMode	bool	Doküman tarzı görünüm
iconSize	QSize	Toolbar ikon boyutu
toolButtonStyle	Qt::ToolButtonStyle	İkon/metin gösterimi
animated	bool	Dock animasyonları

Signals

- `iconSizeChanged(const QSize&)`
- `toolButtonStyleChanged(Qt::ToolButtonStyle)`

Slots / Metotlar

- `setCentralWidget(QWidget*)`, `addToolBar(QToolBar*)`
- `addDockWidget(Qt::DockWidgetArea, QDockWidget*)`
- `setStatusBar(QStatusBar*)`
- `saveState()` / `restoreState(const QByteArray&)`

C++ - OBYS ana pencere iskeleti

```cpp
class AnaPencere : public QMainWindow {
public:
    AnaPencere(QWidget* parent = nullptr) : QMainWindow(parent) {
        auto* merkez = new QTableView(this);
        setCentralWidget(merkez);
        statusBar()->showMessage("Haz r", 2000);
        setWindowTitle("OBYS");
        resize(900, 600);
    }
};
```

Alıştırma

1. `saveState()` hangi bilgileri hatırlar? Pencere boyutunu da hatırlar mı, ayrı mı yönetilir?

Cevabın: _____

2. `centralWidget`'ı bir layout ile doldurmak istediğinde nasıl bir yapı kurarsın? (Doğrudan `setLayout` neden çalışmaz?)

Cevabın: _____

Cevap Anahtarı

1. `saveState()` yalnızca toolbar'ların ve dock widget'ların konumunu, görünürlüğünü ve sırasını hatırlar. Pencerenin boyutu ve ekrandaki konumu için `saveGeometry()` ayrı çağrılır. `restoreState()` ile beraber genellikle `restoreGeometry()` de uygulamanın açılışında çağrılır.
2. `QMainWindow`'a doğrudan layout atanamaz çünkü zaten dahili bir layout (menü/toolbar/dock/status bar yerleşimi) kullanır. Çözüm: bir yardımcı `QWidget` oluştur, ona istediğin layout'u ata ve bu widget'ı `setCentralWidget()` ile merkez yap.

C++ - Merkez widget kalıbı

```
auto* merkez = new QWidget(this);
auto* l = new QVBoxLayout(merkez);
l->addWidget(new QLabel(" st ", merkez));
l->addWidget(new QPushButton("Alt", merkez));
setCentralWidget(merkez);
```

7.2 QMenuBar

QObject → QWidget → QMenuBar

Pencerenin üst kısmında yatay olarak duran menü çubuğudur. Genellikle elle oluşturmazsın; QMainWindow ihtiyaç duyduğunda `menuBar()` çağrısıyla otomatik üretir. İçine QMenu nesneleri eklersin ("Dosya", "Düzen", "Yardım" gibi). Tek bir doğrudan eylem (QAction) de eklenebilir, ancak genel pratik menüleri kullanmaktır. macOS'ta sistem genelinde ekranın üst şeridine taşınan "native" bir menü çubuğu olarak çalışır.

Önemli Properties

Property	Tip	Açıklama
defaultUp	bool	Açılır menüler yukarı doğru mu açılsın?
nativeMenuBar	bool	İşletim sistemi menü çubuğu kullanılsın mı?

Signals

- `triggered(QAction* action)` – bir menü öğesi tıklandı
- `hovered(QAction* action)` – mouse bir öğenin üzerinde

Slots / Metotlar

- `addMenu(const QString& title)` – yeni QMenu döndürür
- `addAction(const QString&)` – doğrudan eylem ekler
- `clear()` – tüm menüleri kaldırır

C++ - OBYS ana menü çubuğu

```
auto* mb = menuBar();                          // QMainWindow'dan al
auto* dosya = mb->addMenu("&Dosya");
dosya->addAction("&Yeni",   this, &AnaPencere::yeniKayit);
dosya->addAction("&Kaydet", this, &AnaPencere::kaydet);
dosya->addSeparator();
dosya->addAction("&    k    ",  qApp, &QApplication::quit);

auto* duzen = mb->addMenu("D& zen ");
duzen->addAction("&Geri Al", this, &AnaPencere::geriAl);

mb->addMenu("&Yard m")->addAction("&Hakk nda",
    this, &AnaPencere::hakkindaGoster);
```

Alıştırma

1. `nativeMenuBar true` (varsayılan) olduğunda macOS'ta menü çubuğu nereye taşınır? Bu hangi durumlarda `false` yapılmalıdır?

Cevabın: _____

2. `addMenu` ile `addAction` arasındaki mantıksal fark nedir? Hangi durumda hangisini tercih edersin?

Cevabın: _____

Cevap Anahtarı

1. macOS'ta `nativeMenuBar` açıkken Qt menü çubuğu pencerenin üstünde değil, ekranın üst şeridindeki sistem menü çubuğunda görüntülenir; bu macOS uygulamalarının beklenen davranışıdır. Tam ekran oyun benzeri uygulamalarda veya menüyü pencerenin içinde tutmak istediğin özel kioskmodlarda `false` yapılır.
2. `addMenu(QString)` bir alt menü (QMenu*) döndürür ve içine eylem eklenir; çoklu seçenekli "Dosya", "Düzen" gibi kategoriler için kullanılır. `addAction` ise tek tıklamalık doğrudan bir eylem ekler. Menü çubuğunda genelde alt menüler kullanılır; tek başına `addAction` nadiren tercih edilir (ör. tek bir global "Yardım" butonu).

7.3 QMenu

QObject → QWidget → QMenu

Açılır (pop-up) menü widget'ıdır. İki tipik kullanımı vardır: **(a)** bir QMenuBar'a alt menü olarak eklenmek, **(b)** sağ tıklama ile çıkan bağlam menüsü (context menu). İçine QAction, alt-QMenu veya ayraçlar eklersin. İç içe menüler oluşturmak için `addMenu` çağrısını özyinelemeli kullanabilirsin. Bağlam menüsü olarak gösterirken `exec()` (bloklayıcı) veya `popup()` (asenkron) metotları arasında seçim yaparsın.

Önemli Properties

Property	Tip	Açıklama
title	QString	Menüyü açan başlık
icon	QIcon	Menü ikonu
tearOffEnabled	bool	Menü "koparılabilir" mi?
separatorsCollapsible	bool	Üst üste ayraçlar gizlensin mi?

Signals

- `triggered(QAction*)` – öğelerden biri seçildi

- `aboutToShow()` / `aboutToHide()` – gösterim öncesi/sonrası
- `hovered(QAction*)` – üzerine gelinen öğe

Slots / Metotlar

- `addAction`, `addMenu`, `addSeparator`
- `exec(const QPoint&)` – modal göster, seçilen QAction* döner
- `popup(const QPoint&)` – modal-olmayan göster

C++ - OBYS tablosunda sağ tık menüsü

```
tablo->setContextMenuPolicy(Qt::CustomContextMenu);
connect(tablo, &QTableView::customContextMenuRequested,
        this, [this, tablo](const QPoint& p) {
    QMenu menu(tablo);
    menu.addAction("D zenle",  this, &AnaPencere::satirDuzenle);
    menu.addAction("Kopyala",  this, &AnaPencere::satirKopyala);
    menu.addSeparator();
    menu.addAction("Sil",      this, &AnaPencere::satirSil);
    menu.exec(tablo->viewport()->mapToGlobal(p));
});
```

Alıştırma

1. `exec()` ile `popup()` arasındaki temel fark nedir? Hangisi hangi kullanım için uygundur?

Cevabın: _____

2. `addSeparator()` ne zaman gereklidir? Görsel olarak ayraç eklemenin kullanılabilirliğe katkısı nedir?

Cevabın: _____

Cevap Anahtarı

1. `exec()` bloklayıcıdır: kullanıcı bir öğe seçene veya menüyü kapatana kadar çağrı geri dönmez ve seçilen `QAction*`'ı (ya da `nullptr`) döndürür; bu yüzden bağlam menülerinde sıkça kullanılır. `popup()` ise menüyü gösterir ve hemen geri döner; sonuç `triggered` sinyali ile asenkron alınır. Olay döngüsünün engellenmesini istemediğin veya menüyü başka bir bağlamdan tetiklediğin durumlarda `popup()` tercih edilir.
2. `addSeparator()` mantıksal olarak farklı gruplardaki eylemleri görsel bir çizgi ile ayırır. Kullanıcının "destructive" (Sil, Sıfırla) eylemleri sıradan olanlardan ayırmasına yardımcı olur ve gözün öğeleri kategorilere ayırmasını kolaylaştırarak hatalı tıklamaları azaltır.

7.4 QToolBar

QObject → QWidget → QToolBar

İkonlu ve hızlı erişim için tasarlanmış bir araç çubuğudur. QMainWindow'a dört kenardan birine tutturulur (üst, alt, sol veya sağ) ve isteğe bağlı olarak kullanıcı tarafından sürüklenip "floating" (yüzen) hale getirilebilir. İçine QAction'lar veya rastgele widget'lar (örn. QComboBox filtre) eklenebilir. `toolButtonStyle` ile yalnızca ikon, yalnızca metin ya da ikon+metin gösterimini seçersin.

Önemli Properties

Property	Tip	Açıklama
movable	bool	Kullanıcı sürükleyebilsin mi?
floatable	bool	Pencereden ayrılabilsin mi?
orientation	Qt::Orientation	Yatay/Dikey
allowedAreas	Qt::ToolBarAreas	Hangi kenarlara yerleşebilir
iconSize	QSize	İkon boyutu
toolButtonStyle	Qt::ToolButtonStyle	İkon/metin gösterimi

Signals

- `actionTriggered(QAction*)`
- `movableChanged(bool)`, `topLevelChanged(bool)`
- `orientationChanged(Qt::Orientation)`

Slots / Metotlar

- `addAction`, `addWidget`, `addSeparator`
- `setOrientation`, `setIconSize`, `setMovable`

C++ - OBYS ana toolbar

```
auto* tb = addToolBar("Ana Araclar");
tb->setIconSize({24, 24});
tb->setToolButtonStyle(Qt::ToolButtonTextUnderIcon);

tb->addAction(QIcon(":/icons/yeni.png"),     "Yeni",
              this, &AnaPencere::yeniKayit);
tb->addAction(QIcon(":/icons/ac.png"),       "A  ",
              this, &AnaPencere::dosyaAc);
tb->addAction(QIcon(":/icons/kaydet.png"),   "Kaydet",
              this, &AnaPencere::kaydet);
tb->addSeparator();
tb->addAction(QIcon(":/icons/yazdir.png"),   "Yazd r",
              this, &AnaPencere::yazdir);
```

Alıştırma

1. `toolButtonStyle` hangi seçenekleri sağlar? En az 3 tanesini yaz ve aralarındaki farkı kısaca açıkla.

Cevabın: _____

2. `allowedAreas` property'sini sınırlamak ne işe yarar? Bir senaryo ile açıkla.

Cevabın: _____

Cevap Anahtarı

1. Başlıca seçenekler:

- `Qt::ToolButtonIconOnly` – yalnızca ikon
- `Qt::ToolButtonTextOnly` – yalnızca metin
- `Qt::ToolButtonTextBesideIcon` – metin ikonun yanında
- `Qt::ToolButtonTextUnderIcon` – metin ikonun altında
- `Qt::ToolButtonFollowStyle` – sistem teması ne diyorsa

İkon-yan-metin masaüstü uygulamalarında en açıklayıcı olanı sayılır; tabletvari arayüzlerde "UnderIcon" tercih edilir.
2. `allowedAreas`, kullanıcının toolbar'ı hangi kenarlara yerleştirebileceğini sınırlar. Örneğin OBYS'de ana araç çubuğunu yalnızca üstte istiyorsan `tb->setAllowedAreas(Qt::TopToolBarArea)` dersin; kullanıcı sürükleyerek sola/sağa taşıyamaz, yine de yüzer (floating) yapabilir. Bu, pencere düzeninin tutarlı kalmasını sağlar.

7.5 QStatusBar

QObject → QWidget → QStatusBar

Pencerenin alt kısmındaki ince bilgi çubuğudur. QMainWindow ihtiyaç duyduğunda `statusBar()` ile otomatik oluşturur. İki çeşit içerik barındırır:

- **Geçici mesajlar:** `showMessage("Kaydedildi", 3000)` gibi belirli bir süre görünür ve kaybolur.
- **Kalıcı widget'lar:** sağ tarafa eklenen göstergeler (saat, kullanıcı adı, satır/sütun bilgisi vb.).

Geçici mesaj gösterilirken kalıcı widget'lar olduğu gibi kalır; yalnızca sol kısımdaki bilgi alanı geçici olarak değişir.

Önemli Properties

Property	Tip	Açıklama
sizeGripEnabled	bool	Sağ alt köşede boyutlandırma tutamacı

Signals

- `messageChanged(const QString& message)` – gösterilen geçici mesaj değişti

Slots / Metotlar

- `showMessage(const QString&, int timeout = 0)`
- `clearMessage()`
- `addWidget(QWidget*, int stretch = 0)` – sol taraf, geçici mesajla yer değiştirir
- `addPermanentWidget(QWidget*, int stretch = 0)` – sağ taraf, hep görünür

C++ - OBYS durum çubuğu

```
auto* sb = statusBar();
sb->showMessage("Haz r", 2000);

// Sag tarafta kalici saat etiketi
auto* saat = new QLabel(sb);
sb->addPermanentWidget(saat);
auto* zamanlayici = new QTimer(this);
connect(zamanlayici, &QTimer::timeout, this, [saat] {
    saat->setText(QTime::currentTime().toString("HH:mm:ss"));
});
zamanlayici->start(1000);

// Bir islem sonrasi gecici mesaj
sb->showMessage("Kaydedildi", 3000);
```

Alıştırma

1. `showMessage`'ın `timeout` parametresi ne anlama gelir? `0` verirsen ne olur?

Cevabın: _____

2. `addWidget` ile `addPermanentWidget` arasındaki davranış farkı nedir? Hangi tür bilgileri hangisine koyarsın?

Cevabın: _____

Cevap Anahtarı

1. `timeout` milisaniye cinsinden mesajın ne kadar görüneceğidir. `0` (varsayılan) verildiğinde mesaj süresiz olarak kalır; ancak `clearMessage()` çağrılana veya yeni bir `showMessage` ile üzerine yazılana kadar durur.
2. `addWidget` sol tarafa eklenir ve geçici mesaj gösterildiğinde geçici olarak gizlenir; bu yüzden "geçişlerle değişebilen" bilgiler için uygundur. `addPermanentWidget` ise sağ tarafa eklenir ve geçici mesajdan etkilenmez; saat, internet bağlantı simgesi, satır/ sütun göstergesi gibi kullanıcının her an görmek isteyeceği bilgiler için kullanılır.

7.6 QDockWidget

QObject → QWidget → QDockWidget

Yüzebilen veya pencereye tutturulabilen yardımcı paneldir. IDE'lerde gördüğün "Outline", "Messages", "Project Explorer" gibi pencereler birer dock widget'tır. Kullanıcı sürükleyerek başka bir kenara taşıyabilir, pencereden tamamen ayırarak bağımsız bir pencere yapabilir veya kapatabilir. `features` bayrakları ile bu davranışları sınırlayabilirsin. İçinde tek bir "payload" widget tutar; karmaşık panelleri bir konteyner widget üzerinden eklersin.

Önemli Properties

Property	Tip	Açıklama
floating	bool	Pencereden ayrı mı?
features	QDockWidget::DockWidgetFeatures	Movable/Floatable/Closable bayrakları
allowedAreas	Qt::DockWidgetAreas	Hangi kenarlara tutunabilir
titleBarWidget	QWidget*	Özel başlık çubuğu
windowTitle	QString	Başlıkta görünen ad

Signals

- `topLevelChanged(bool floating)`
- `dockLocationChanged(Qt::DockWidgetArea)`
- `visibilityChanged(bool)`

Slots / Metotlar

- `setWidget(QWidget*)` – içerik widget'ını belirle
- `setFloating(bool)`
- `setAllowedAreas(Qt::DockWidgetAreas)`

C++ - OBYS arama paneli

```cpp
auto* dock = new QDockWidget("Arama", this);
dock->setAllowedAreas(Qt::LeftDockWidgetArea
                      | Qt::RightDockWidgetArea);
dock->setFeatures(QDockWidget::DockWidgetMovable
                  | QDockWidget::DockWidgetFloatable);

auto* icerik = new QWidget(dock);
auto* l = new QVBoxLayout(icerik);
l->addWidget(new QLineEdit(icerik));
l->addWidget(new QListWidget(icerik));
dock->setWidget(icerik);

addDockWidget(Qt::LeftDockWidgetArea, dock);
```

Alıştırma

1. `features` bayraklarının (`Movable`, `Floatable`, `Closable`, `VerticalTitleBar`) kombinasyonu nasıl çalışır? Bir dock'u sabit ama gizlenebilir yapmak için hangi bayrakları kullanırsın?

Cevabın: _____

2. `setWidget` ile içine yalnızca bir tek widget koyabilirsin. Birden fazla görsel öğe eklemek istersen nasıl bir yapı kurarsın?

Cevabın: _____

Cevap Anahtarı

1. `features`, `|` operatörü ile birleştirilen bir bayrak kümesidir. `Movable` kullanıcının sürükleyerek başka kenara taşımasını, `Floatable` pencereden tamamen ayırmasını, `Closable` "X" butonuyla gizlemesini sağlar. "Sabit ama gizlenebilir" istiyorsan `QDockWidget::DockWidgetClosable` bayrağını tek başına verirsin (`Movable` ve `Floatable` olmadan); böylece yeri sabit, fakat kullanıcı kapatabilir.
2. Bir konteyner widget (örn. yeni bir `QWidget`) oluşturup ona istediğin layout'u (`QVBoxLayout`, `QGridLayout` vb.) atarsın, çocukları bu layout'a eklersin ve nihayetinde `dock->setWidget(konteyner)` çağırırsın. Dock widget bu konteyneri tek bir öğe olarak görür.

7.7 QSystemTrayIcon

QObject → QSystemTrayIcon

İşletim sisteminin sistem tepsisinde (Windows: saatin yanı, macOS: menü çubuğu sağı, Linux: panel köşesi) görünen küçük ikondur. Arka planda çalışan uygulamalar (yedekleme aracı, mesajlaşma istemcisi, OBYS

bildirimcisi) için idealdir. QWidget değildir; bir QObject'tir, bu nedenle layout'a eklenmez. Tıklamada bir QMenu açabilir ve OS-yerel bildirim balonları (`showMessage`) gösterebilir.

Önemli Properties

Property	Tip	Açıklama
icon	QIcon	Tepside görünen ikon
toolTip	QString	Üstüne gelince çıkan ipucu
visible	bool	Görünür mü?
contextMenu	QMenu*	Sağ tık menüsü

Signals

- `activated(QSystemTrayIcon::ActivationReason)`
- `messageClicked()` – bildirim balonuna tıklandı

Slots / Metotlar

- `show()`, `hide()`
- `showMessage(title, msg, icon, timeoutMs)`
- `setContextMenu(QMenu*)`

C++ - OBYS tepsi ikonu

```cpp
auto* tray = new QSystemTrayIcon(QIcon(":/icons/obys.png"), this);
tray->setToolTip("OBYS - Arkaplanda calisiyor");

auto* menu = new QMenu(this);
menu->addAction("Pencereyi Goster",
                this, &QWidget::showNormal);
menu->addSeparator();
menu->addAction("Cikis", qApp, &QApplication::quit);
tray->setContextMenu(menu);

connect(tray, &QSystemTrayIcon::activated,
        this, [this](QSystemTrayIcon::ActivationReason r) {
    if (r == QSystemTrayIcon::Trigger) showNormal();
});

tray->show();
tray->showMessage("OBYS", "Yedekleme tamamlandi.",
                  QSystemTrayIcon::Information, 3000);
```

Alıştırma

1. `ActivationReason` hangi sabitleri içerir (en az 3 tane)? `Trigger` ile `DoubleClick` arasındaki fark nedir?

Cevabın: _____

2. `showMessage()` ile `QMessageBox::information()` arasındaki temel fark nedir? Hangi durumda hangisini tercih edersin?

Cevabın: _____

Cevap Anahtarı

1. Başlıca sabitler:

- `QSystemTrayIcon::Unknown`
- `QSystemTrayIcon::Context` – sağ tık (menü açılır)
- `QSystemTrayIcon::Trigger` – tek sol tık
- `QSystemTrayIcon::DoubleClick` – çift sol tık
- `QSystemTrayIcon::MiddleClick` – orta tık

`Trigger` kullanıcının ikona tek tıkladığında üretilir; genellikle pencereyi göstermek için kullanılır. `DoubleClick` ise çift tıkta üretilir. Aynı sırayla iki sinyalin de üretilebileceğine dikkat et; davranışı ona göre tasarla.
2. `tray->showMessage()` işletim sisteminin yerel bildirim balonunu kullanır; modal değildir, kullanıcının işini bölmez ve birkaç saniye sonra kaybolur. Arkaplan bildirimleri ("Yedekleme tamamlandı") için uygundur. `QMessageBox::information()` ise modal bir diyalog açar ve kullanıcı kapatana kadar uygulamayı bekletir; mutlaka karşılığı alınması gereken kritik bilgiler için kullanılır.

7.8 QWizard

QObject → QWidget → QDialog → QWizard

Adım adım form sihirbazıdır. Her adım bir QWizardPage nesnesidir; "İleri" / "Geri" / "Bitir" butonları otomatik olarak çubuğa yerleşir. Sayfalar arasında veri taşımak için `registerField` mekanizması kullanılır: bir sayfada "alanı" kaydedersin, sonraki sayfalarda `field("ad")` ile okursun. Kurulum sihirbazları, yeni proje oluşturma ekranları, çok adımlı kayıt formları için tipik kullanım alanıdır.

Önemli Properties

Property	Tip	Açıklama
options	QWizard::WizardOptions	Görsel/akış seçenekleri
titleFormat	Qt::TextFormat	Başlık biçimi
subTitleFormat	Qt::TextFormat	Alt başlık biçimi
startId	int	Başlangıç sayfası ID'si

Signals

- `currentIdChanged(int id)`
- `pageAdded(int id)` / `pageRemoved(int id)`
- `helpRequested()`

Slots / Metotlar

- `addPage(QWizardPage*)` / `setPage(int, QWizardPage*)`
- `removePage(int)`, `next()`, `back()`, `restart()`
- `registerField(name, widget, property, changedSignal)`

C++ - OBYS kurulum sihirbazı

```
QWizard sihirbaz;
sihirbaz.setWindowTitle("OBYS Kurulum");

auto* hosgeldin = new QWizardPage;
hosgeldin->setTitle("Ho  Geldin");
auto* hl = new QVBoxLayout(hosgeldin);
hl->addWidget(new QLabel("OBYS kurulumuna ho  geldin."));
sihirbaz.addPage(hosgeldin);

auto* ayarlar = new QWizardPage;
ayarlar->setTitle("Ayarlar");
auto* dbEdit = new QLineEdit;
auto* al = new QFormLayout(ayarlar);
al->addRow("Veritabani yolu:", dbEdit);
ayarlar->registerField("dbYolu*", dbEdit);  // * = zorunlu
sihirbaz.addPage(ayarlar);

auto* bitir = new QWizardPage;
bitir->setTitle("Bitir");
auto* bl = new QVBoxLayout(bitir);
bl->addWidget(new QLabel("Kurulum tamamlanmaya haz r."));
sihirbaz.addPage(bitir);

if (sihirbaz.exec() == QDialog::Accepted) {
    QString yol = sihirbaz.field("dbYolu").toString();
    qDebug() << "Secilen yol:" << yol;
}
```

Alıştırma

1. `registerField` ne işe yarar? Alan adının sonuna eklenen `*` karakterinin anlamı nedir?

Cevabın: _____

2. `QWizardPage::isComplete()`'ı override etmek hangi davranışı kontrol eder? Bir kullanım senaryosu yaz.

Cevabın: _____

Cevap Anahtarı

1. `registerField`, bir widget'ın bir property'sini sihirbaz çapında bir "alan" (field) olarak kaydeder; bu sayede başka sayfalar `field("ad")` ile değeri okuyabilir ve sihirbaz kapanırken birikmiş tüm veriye erişebilirsin. Alan adının sonuna `*` eklenirse o alan "zorunlu" sayılır; boş bırakıldığı sürece "İleri" butonu pasif kalır.
2. `isComplete()` sayfanın geçerli/eksiksiz olup olmadığını döndürür. `true` olduğunda "İleri" butonu aktif olur. Override ederek özel doğrulama yazabilirsin: örneğin OBYS kurulum sayfasında girilen veritabanı yolunun *var olan bir klasör* olduğunu kontrol eder, geçerli ise `true` dönersin. Değişiklikleri yayınlamak için `completeChanged()` sinyalini emit etmeyi unutma.

7.9 QMdiArea

QObject → QWidget → QFrame → QAbstractScrollArea → QMdiArea

Çoklu Belge Arayüzü (Multiple Document Interface) konteyneridir. İçinde birden fazla QMdiSubWindow barındırır; her alt pencere bağımsız olarak taşınabilir, küçültülebilir, büyütülebilir. Aynı uygulamada birkaç dosyayı yan yana açmak için klasik bir yaklaşımdır. `viewMode` ile iki farklı görünüm arasında geçiş yapılır:

`SubWindowView` (klasik kayan alt pencereler) ve `TabbedView` (sekmeli arayüz). OBYS'de birden çok `.obys` dosyasını aynı anda görüntülemek istediğinde kullanılabilir.

Önemli Properties

Property	Tip	Açıklama
viewMode	QMdiArea::ViewMode	SubWindow / Tabbed
activationOrder	QMdiArea::WindowOrder	Pencere geçiş sırası
tabsClosable	bool	Sekmelerde X butonu
tabsMovable	bool	Sekmeler sürüklenebilir mi?
documentMode	bool	Doküman tarzı sekme

Signals

- `subWindowActivated(QMdiSubWindow* window)`

Slots / Metotlar

- `addSubWindow(QWidget*, Qt::WindowFlags = ...)`
- `removeSubWindow(QWidget*)`
- `cascadeSubWindows()`, `tileSubWindows()`
- `closeActiveSubWindow()`, `closeAllSubWindows()`

C++ - Birden fazla OBYS dosyasi

```cpp
auto* mdi = new QMdiArea(this);
setCentralWidget(mdi);
mdi->setViewMode(QMdiArea::TabbedView);
mdi->setTabsClosable(true);
mdi->setTabsMovable(true);

auto belgeAc = [mdi](const QString& yol) {
    auto* duzenleyici = new QPlainTextEdit;
    duzenleyici->setWindowTitle(QFileInfo(yol).fileName());
    auto* alt = mdi->addSubWindow(duzenleyici);
    alt->show();
};
belgeAc("ogrenciler_2024.obys");
belgeAc("notlar_bahar.obys");

mdi->tileSubWindows();    // Yan yana diz
```

Alıştırma

1. `SubWindowView` ile `TabbedView` arasında program çalışırken kullanıcı / kod nasıl geçiş yapar? İki modun kullanım senaryosunu kısaca karşılaştır.

Cevabın: _____

2. `cascadeSubWindows()` ve `tileSubWindows()` ne yapar? Aralarındaki fark nedir?

Cevabın: _____

Cevap Anahtarı

1. Geçiş kod tarafından `mdi->setViewMode(QMdiArea::TabbedView)` (veya `SubWindowView`) ile yapılır; kullanıcıya doğrudan bir arayüz sunulmaz, genellikle bir "Görünüm" menüsünden tetiklenir. `SubWindowView` klasik MDI deneyimidir (serbestçe sürüklenebilir alt pencereler) ve aynı anda birden fazla belgeyi görsel olarak karşılaştırmak için iyidir. `TabbedView` her belgeyi bir sekmeye atar; modern editör benzeri deneyim için tercih edilir, küçük ekranlarda daha kullanışlıdır.
2. `cascadeSubWindows()` alt pencereleri diyagonal olarak üst üste, hafif kaydırılmış şekilde dizer ("catlak" efekti); başlık çubukları görünür, içerikler kısmen kapalıdır. `tileSubWindows()` ise alt pencereleri mevcut alanı paylaşacak şekilde yan yana / üst alta döşer (grid). İlki "üst üste yığ" demektir, ikincisi "yan yana diz" demektir.

Kategori 07 Bitiş Quizi

Pencere Elemanları – Mini Quiz

1. (Kavramsal) QMainWindow'un `menuBar()`'ı ilk çağrıldığında geri dönen pointer `nullptr` olabilir mi? Eğer pencereye henüz hiçbir menü eklemediysen ne dönmesini beklersin? Açıkla.

Cevabın: _____

2. (Eşleştirme) Aşağıdaki senaryoları en uygun widget ile eşleştir:

- Sürekli görünür bilgi göstergesi (saat, kullanıcı adı) → _____
- Arka planda çalışan uygulamanın işletim sistemi ikonu → _____
- Pencereye gömülebilen, yüzebilen yardımcı panel → _____

3. (Kod okuma) Aşağıdaki toolbar kodunda kaç tane ayraç (separator) görünür?

C++ - Soru 3

```
toolbar->addAction(acYeni);
toolbar->addSeparator();
toolbar->addAction(acKaydet);
toolbar->addAction(acFarkliKaydet);
toolbar->addSeparator();
toolbar->addAction(acCikis);
```

Cevabın: _____

4. (Kavramsal) QWizard'da bir sayfada girilen değeri kayıt edip sonraki sayfalardan okumak için hangi API kullanılır? Hem kayıt hem okuma için kullanılan iki metodu yaz.

Cevabın: _____

5. (Mikro kod) `QSystemTrayIcon* tray` olduğunu varsay. Sistem tepsisinde *"OBYS"* başlıklı, *"Yeni duyuru var."* mesajlı, *3 saniye* süren bilgi tipli bir bildirim gösteren tek satır kodu yaz.

Kategori 07 Quiz – Cevap Anahtarı

1. Hayır, `nullptr` dönmez. QMainWindow'un `menuBar()` metodu "lazy" (tembel) çalışır: ilk çağrıda otomatik olarak yeni bir QMenuBar oluşturup pencereye takar ve onu döndürür. Bu yüzden hiç menü eklemesen bile çağrı geçerli bir pointer döner. Aynı durum `statusBar()` için de geçerlidir.
2. Eşleştirme:

- Sürekli görünür gösterge → QStatusBar (`addPermanentWidget`)
- Arka plan uygulama ikonu → QSystemTrayIcon
- Yüzen / tutturulabilir yardımcı panel → QDockWidget

3. 2 ayraç görünür. Kodda iki kez `addSeparator()` çağrılmış, ikisi de farklı bağlamda (eylemler arasında) olduğu için `separatorsCollapsible` default davranışı tarafından gizlenmez. Eğer iki `addSeparator` arka arkaya çağrılsaydı (arada eylem olmadan), Qt onları tek bir görünür ayraca sıkıştırırdı.
4. `registerField(name, widget, ...)` ile sayfada alan kayıt edilir; sonraki sayfalardan `wizard()->field("name")` (veya wizard nesnesi üzerinden `field("name")`) ile okunur. Alan adına `*` eklenirse zorunlu olur.
5. Tek satır:

C++ - Cevap 5

```cpp
tray->showMessage("OBYS", "Yeni duyuru var.",
                  QSystemTrayIcon::Information, 3000);
```

8 Diyaloglar

Kullanıcıyla kısa etkileşim kurmak için açılan pencereler. Generic QDialog tabanı + Qt'nin hazır standart diyalogları: QMessageBox (bilgi/uyarı), QFileDialog (dosya seç), QColorDialog (renk), QFontDialog (font), QInputDialog (tek değer giriş), QProgressDialog (ilerleme), QErrorMessage (hata günlüğü). OBYS'de QFileDialog ile `.obys` açma/kaydetme (*(bkz. kitap)*) yaygın kullanılır.

8.1 QDialog

QObject → QWidget → QDialog

OBYS'de "Not Girişi" veya "Yeni Öğrenci" formu açıldığında karşına çıkan pencere bir QDialog'dur. Tüm özel diyalogların temel sınıfıdır. *Modal* (kullanıcı kapatana kadar arka plan pasif) veya *modeless* (yan yana kullanılır) çalışabilir. exec() ile kendi event loop'unu başlatır ve kullanıcı accept() veya reject() çağırana kadar bloklar.

Önemli Properties

Property	Tip	Açıklama
modal	bool	Açıldığında modal mi olacak
result	int	accept/reject sonucu (Accepted/Rejected)
sizeGripEnabled	bool	Sağ alt köşe boyut tutamacı
windowTitle	QString	Pencere başlığı

Signals

- `accepted()` – accept çağrıldığında
- `rejected()` – reject çağrıldığında
- `finished(int result)` – Diyalog kapandığında (her iki durumda)

Slots

- `accept()` – Diyalogu Accepted (1) sonucuyla kapatır
- `reject()` – Diyalogu Rejected (0) sonucuyla kapatır
- `done(int)` – Belirli sonuçla kapatır
- `open()` – Modal aç (event loop bloklanmaz)
- `exec()` – Modal aç ve sonucu döndür

C++ - Mini ornek: OBYS NotGirisDialog iskeleti

```
class NotGirisDialog : public QDialog {
    Q_OBJECT
public:
    explicit NotGirisDialog(QWidget* parent = nullptr)
        : QDialog(parent) {
        setWindowTitle(tr("Not Girisi"));
        auto* form = new QFormLayout(this);
        m_not = new QSpinBox(this);
        form->addRow(tr("Not:"), m_not);
        auto* btns = new QDialogButtonBox(
            QDialogButtonBox::Ok | QDialogButtonBox::Cancel, this);
```

```
12          form->addRow(btns);
13          connect(btns, &QDialogButtonBox::accepted, this,
               &QDialog::accept);
14          connect(btns, &QDialogButtonBox::rejected, this,
               &QDialog::reject);
15      }
16      int notDegeri() const { return m_not->value(); }
17  private:
18      QSpinBox* m_not;
19  };
20
21  NotGirisDialog dlg(this);
22  if (dlg.exec() == QDialog::Accepted) { kaydet(dlg.notDegeri()); }
```

Alıştırma

1. exec() ile show() arasındaki temel fark nedir? Hangisi event loop bloklar?

Cevabın: _____

2. accept() ve reject() çağrıldığında exec() sırasıyla hangi int değerleri döndürür?

Cevabın: _____

Cevap Anahtarı

1. exec() kendi yerel event loop'unu başlatır ve diyalog kapanana kadar çağıran kodu *bloklar*; modal çalışmanın klasik yoludur. show() ise diyalogu görünür yapar ve hemen geri döner; bloklamaz, modeless kullanım için uygundur (open() bloklamayan modal varyanttır).
2. accept() sonra exec() `QDialog::Accepted` (1) döner; reject() sonra exec() `QDialog::Rejected` (0) döner.

8.2 QMessageBox

QObject → QWidget → QDialog → QMessageBox

Standart *bilgi*, *uyarı*, *hata* ve *soru* diyaloğu. OBYS'de "Kayıt başarılı", "Kaydedilmemiş değişiklikler var, çıkmak istediğine emin misin?" veya "Bağlantı koptu" gibi mesajları tek satırda göstermek için statik metodları kullanırız. İkonu, butonları ve metin bileşenleri (kısa metin + açıklama + detay) hazır gelir.

Önemli Properties

Property	Tip	Açıklama
text	QString	Ana mesaj satırı
informativeText	QString	Açıklayıcı ek metin
detailedText	QString	"Show Details" altındaki uzun metin
icon	Icon	Information / Warning / Critical / Question
standardButtons	StandardButtons	Ok, Cancel, Yes, No, Save, Discard...
defaultButton	QPushButton*	Enter'a basıldığında tetiklenen

Static Metotlar (en sık kullanım)

- `QMessageBox::information(parent, baslik, metin)`
- `QMessageBox::warning(parent, baslik, metin)`
- `QMessageBox::critical(parent, baslik, metin)`

- `QMessageBox::question(parent, baslik, metin, butonlar, varsayilan)`
- `QMessageBox::about(parent, baslik, metin)`

Signals

- `buttonClicked(QAbstractButton* button)` – Bir butona tıklandığında

C++ - Mini ornek: OBYS kaydedilmemis degisiklik onayi

```cpp
auto cevap = QMessageBox::question(this,
    tr("Cikis"),
    tr("Kaydedilmemis degisiklikler var.\nYine de cikilsin mi?"),
    QMessageBox::Save | QMessageBox::Discard | QMessageBox::Cancel,
    QMessageBox::Save);

switch (cevap) {
    case QMessageBox::Save:    dosyayiKaydet(); event->accept(); break;
    case QMessageBox::Discard: event->accept(); break;
    default:                   event->ignore(); break;
}
```

Alıştırma

1. QMessageBox::information ile QMessageBox::question arasındaki temel fark nedir? (İkon ve dönüş değeri açısından)

Cevabın: _____

2. `detailedText` kullanıcıya nasıl gösterilir? Diyalog ilk açıldığında doğrudan görünür mü?

Cevabın: _____

Cevap Anahtarı

1. information bilgi ikonu (i) gösterir ve genelde sadece *Ok* butonu içerir; dönüş değeri çoğunlukla göz ardı edilir. question soru ikonu (?) gösterir, varsayılan olarak Yes/No butonları içerir ve *kullanıcının seçimini* (`StandardButton` enum) döndürür; akış kontrolü için kullanılır.
2. `detailedText` ilk açılışta gizlidir. Diyalogun alt kısmında otomatik olarak beliren "Show Details..." butonuna tıklayınca aşağıda kaydırılabilir bir metin alanı açılır. Hata ayıklama bilgisi (stack trace, log) eklemek için idealdir.

8.3 QFileDialog

QObject → QWidget → QDialog → QFileDialog

Dosya veya dizin seçtirmek için kullanılan standart işletim sistemi diyaloğu. OBYS'de `.obys` dosyasını açıp/kaydederken (*(bkz. kitap)*) en sık kullanılan diyalogdur. Statik metotlarla tek satırda; örnek vermek gerekirse getOpenFileName, getSaveFileName ve getExistingDirectory en yaygın kullanım şekilleridir.

Önemli Properties

Property	Tip	Açıklama
fileMode	FileMode	AnyFile, ExistingFile, ExistingFiles, Directory
viewMode	ViewMode	Detail veya List
nameFilters	QStringList	"OBYS (*.obys);;Tum (*)"
defaultSuffix	QString	Kullanıcı uzantı yazmazsa eklenir (`obys`)
acceptMode	AcceptMode	AcceptOpen veya AcceptSave

Static Metotlar

- `getOpenFileName(parent, baslik, dir, filtre)` → QString
- `getSaveFileName(parent, baslik, dir, filtre)` → QString
- `getOpenFileNames(...)` → QStringList (çoklu)
- `getExistingDirectory(parent, baslik, dir)` → QString

Signals

- `fileSelected(const QString&)` – Tek dosya seçildi
- `filesSelected(const QStringList&)` – Birden çok dosya
- `filterSelected(const QString&)` – Filtre değişti

C++ - Mini ornek: OBYS .obys dosyasi acma

```cpp
QString yol = QFileDialog::getOpenFileName(this,
    tr("OBYS Dosyasi Ac"),
    QDir::homePath(),
    tr("OBYS Dosyalari (*.obys);;Tum Dosyalar (*)"));

if (!yol.isEmpty()) {
    QFile f(yol);
    if (f.open(QIODevice::ReadOnly)) {
        QDataStream in(&f);
        in >> m_okul;  // Okul nesnesi yuklenir
    }
}
```

Alıştırma

1. getOpenFileName statik metoduyla, `fileMode ExistingFile` olarak ayarlanmış bir QFileDialog nesnesi arasında işlevsel olarak nasıl bir ilişki vardır?

Cevabın: _____

2. `defaultSuffix` property'si ne işe yarar? OBYS kaydetme diyaloğunda neden `obys` olarak ayarlanmalı?

Cevabın: _____

Cevap Anahtarı

1. getOpenFileName aslında arka planda tam olarak `fileMode = ExistingFile` ve `acceptMode = AcceptOpen` ayarlanmış bir QFileDialog oluşturup exec() eden bir kolaylık sarmalayıcısıdır. Dönüş tek dosya yoludur. Daha çok özelleştirme istiyorsan (özel butonlar, ön izleme) doğrudan QFileDialog nesnesi oluşturup property'leri kendin ayarlarsın.
2. `defaultSuffix`, kullanıcı dosya adına uzantı yazmazsa (örnek: "yedek") otomatik olarak `.obys` ekler. Böylece OBYS dışında bir uzantıyla kaydedilip sonra tanınmayan dosyalar oluşmaz; open() sırasında filtre tutarlılığı korunur.

8.4 QColorDialog

QObject → QWidget → QDialog → QColorDialog

Renk seçici standart diyalog. Kullanıcı bir paletten veya HSV/RGB değerleri girerek renk seçer. OBYS'de "Karne grafiği arka plan rengi" veya "Tema vurgu rengi" seçtirmek için kullanabilirsin. getColor statik metodu en pratik kullanım şeklidir.

Önemli Properties

Property	Tip	Açıklama
currentColor	QColor	Şu anda seçili renk
options	ColorDialogOptions	ShowAlphaChannel, NoButtons, DontUseNativeDialog

Static Metotlar

- `QColor QColorDialog::getColor(const QColor& initial = Qt::white, QWidget* parent = nullptr, const QString& title = QString(), ColorDialogOptions options = ColorDialogOptions()`

Signals

- `colorSelected(const QColor&)` – Ok'a basıldığında
- `currentColorChanged(const QColor&)` – Renk her değiştiğinde (canlı önizleme)

C++ - Mini ornek: OBYS karne grafigi rengi

```cpp
QColor secilen = QColorDialog::getColor(
    m_grafikRengi,              // baslangic
    this,                       // parent
    tr("Grafik Rengi Sec"),
    QColorDialog::ShowAlphaChannel);

if (secilen.isValid()) {
    m_grafikRengi = secilen;
    m_grafikWidget->update();
}
```

Alıştırma

1. `QColorDialog::ShowAlphaChannel` bayrağı ne için kullanılır? OBYS'de hangi senaryoda yararlı olur?

Cevabın: _____

2. `currentColorChanged` sinyali ne zaman yayınlanır? Bunu `colorSelected`'ten ayıran nedir?

Cevabın: _____

Cevap Anahtarı

1. `ShowAlphaChannel` diyaloğa alfa (saydamlık) kaydırıcısı ekler. Böylece kullanıcı sadece RGB değil, opaklığı da seçebilir. OBYS'de örneğin not dağılım grafiğinde sütunların yarı saydam çizilmesini istiyorsan (alttaki kılavuz çizgileri görünsün diye) buna ihtiyacın olur.
2. `currentColorChanged`, kullanıcı palette gezindikçe veya HSV kaydırıcılarını oynattıkça *her renk değişiminde* anlık olarak yayınlanır; canlı önizleme yapmak için kullanılır. `colorSelected` ise yalnızca kullanıcı *Ok* butonuna basıp seçimi onayladığında bir kez yayınlanır.

8.5 QFontDialog

QObject → QWidget → QDialog → QFontDialog

Font ailesini, stilini ve boyutunu seçtirmek için kullanılan standart diyalog. OBYS'de "Rapor başlığı fontu" veya "Yazdırma çıktısı yazı tipi" gibi ayarlarda kullanışlıdır. Tıpkı QColorDialog gibi, statik getFont metodu en yaygın kullanım şeklidir.

Önemli Properties

Property	Tip	Açıklama
currentFont	QFont	Şu anda seçili font
options	FontDialogOptions	NoButtons, DontUseNativeDialog, vb.

Static Metotlar

- `QFont QFontDialog::getFont(bool* ok, const QFont& initial, QWidget* parent = nullptr, const QString& title = QString())`

Signals

- `fontSelected(const QFont&)` – Ok'a basıldığında
- `currentFontChanged(const QFont&)` – Font önizlemesi her değiştiğinde

C++ - Mini ornek: OBYS rapor fontu secimi

```cpp
bool ok = false;
QFont yeni = QFontDialog::getFont(&ok,
    m_raporFontu,            // baslangic
    this,
    tr("Rapor Fontu Sec"));

if (ok) {
    m_raporFontu = yeni;
    m_onizlemeLabel->setFont(yeni);
}
// ok == false ise kullanici Cancel'a bast , secimi yok say.
```

Alıştırma

1. getFont'un `bool* ok` parametresi ne anlam taşır? Neden dönüş değerinin (QFont) yanında ek olarak gerek vardır?

Cevabın: _____

2. Diyalogun preview alanını canlı izlemek (kullanıcı font seçerken arka plandaki bir label'ı eş zamanlı güncellemek) için hangi sinyali, hangi diyalog kullanım şekliyle bağlardın?

Cevabın: _____

Cevap Anahtarı

1. QFont her zaman geçerli bir nesne döndürür; kullanıcı Cancel bassa bile bir font (genelde başlangıç fontu) döner. Bu yüzden "kullanıcı gerçekten seçimi onayladı mı?" bilgisini ayrıca tutmak gerekir. `ok`, çağrı tamamlanınca `true` ise Ok, `false` ise Cancel anlamına gelir.
2. Statik getFont bunun için uygun değildir; çünkü diyalog kapanınca tek seferde sonucu döndürür. Bunun yerine `auto* dlg = new QFontDialog(this);` ile bir nesne oluşturup `connect(dlg, &QFontDialog::currentFontChanged, label, &QLabel::setFont);` bağlar, sonra `dlg->open()` veya `dlg->exec()` çağırırsın. Böylece kullanıcı her seçim değiştirdiğinde label anında güncellenir.

8.6 QInputDialog

QObject → QWidget → QDialog → QInputDialog

Tek bir değer (metin, tam sayı, ondalıklı sayı veya listeden öğe) girmek için kullanılan basit ama güçlü diyalog. OBYS'de "Yeni ders kodu", "Yeni şube adı" veya "Geçme notu eşiği" gibi küçük girişler için tam bir form yazmak yerine QInputDialog statik metotlarıyla tek satırda halledebilirsin.

Static Metotlar

- `getText(parent, baslik, etiket, echo, baslangic, *ok)` → QString
- `getInt(parent, baslik, etiket, value, min, max, step, *ok)` → int
- `getDouble(parent, baslik, etiket, value, min, max, decimals, *ok)` → double
- `getItem(parent, baslik, etiket, items, current, editable, *ok)` → QString
- `getMultiLineText(parent, baslik, etiket, baslangic, *ok)` → QString

Önemli Properties (nesne kullanımı)

Property	Tip	Açıklama
inputMode	InputMode	TextInput, IntInput, DoubleInput
labelText	QString	Üstteki açıklama
textValue	QString	Şu anki metin değeri
intValue	int	Şu anki int değeri

Signals

- `textValueChanged(const QString&)`
- `intValueChanged(int)`
- `doubleValueChanged(double)`

C++ - Mini ornek: OBYS yeni ders kodu

```cpp
bool ok = false;
QString kod = QInputDialog::getText(this,
    tr("Yeni Ders"),
    tr("Ders kodu (orn: BILB2000):"),
    QLineEdit::Normal,
    QString(),
    &ok);

if (ok && !kod.isEmpty()) {
    m_okul.dersEkle(kod);
}
```

Alıştırma

1. getInt'in `step` parametresi ne için kullanılır? Hangi UI bileşeniyle ilgilidir?

Cevabın: _____

2. Kullanıcı diyalogda Cancel butonuna bastığında `*ok` parametresinin değeri ne olur? Bu durumda dönen QString (veya int) ne içerir?

Cevabın: _____

Cevap Anahtarı

1. getInt arka planda bir QSpinBox oluşturur. `step`, bu spin box'ın yukarı/aşağı oklarına basıldığında değerin kaçar kaçar artıp azalacağını belirler. Örneğin geçme notu eşiği için `step = 5` yapmak kullanıcıya 50, 55, 60, 65... gibi pratik adımlar sunar.
2. Cancel'a basılırsa `*ok = false` olur. Dönen değer ise tipine göre "boş" kabul edilebilecek bir değerdir (QString için boş string `""`, int için verilen `value` başlangıç değeri). Bu yüzden mutlaka önce `ok` kontrol edilmeli, ardından dönen değer kullanılmalıdır.

8.7 QProgressDialog

QObject → QWidget → QDialog → QProgressDialog

Uzun süren bir işlem (toplu rapor üretimi, dosya içe aktarma, ağdan veri çekme) sırasında ilerleme yüzdesini ve İptal butonunu gösteren özel diyalog. OBYS'de tüm öğrencilerin karne PDF'sini üretirken kullanıcıya hem hangi aşamada olduğunu hem de gerekirse iptal etme imkânını sunmak için idealdir. *Otomatik görünürlük* özelliği vardır: işlem hızlıca biterse diyalog hiç açılmaz.

Önemli Properties

Property	Tip	Açıklama
minimum	int	İlerleme alt sınırı (genelde 0)
maximum	int	İlerleme üst sınırı (örn: 100)
value	int	Şu anki ilerleme
labelText	QString	Üstteki açıklama metni
cancelButtonText	QString	İptal butonu yazısı
minimumDuration	int	Diyalog kaç ms sonra açılsın (varsayılan 4000)
autoClose	bool	maximum'a ulaşınca otomatik kapan
autoReset	bool	maximum'a ulaşınca value'yi sıfırla

Signals

- `canceled()` – Kullanıcı İptal'e bastığında

Slots

- `setValue(int)` – İlerlemeyi günceller
- `setLabelText(const QString&)` – Metni günceller
- `cancel()` – Programatik iptal
- `reset()` – Diyalogu kapatıp sıfırlar

C++ - Mini ornek: OBYS toplu karne uretimi

```
QProgressDialog dlg(tr("Karneler uretiliyor..."),
                    tr("Iptal"), 0, ogrenciler.size(), this);
dlg.setWindowModality(Qt::WindowModal);
dlg.setMinimumDuration(500);

for (int i = 0; i < ogrenciler.size(); ++i) {
    if (dlg.wasCanceled()) break;
    dlg.setValue(i);
    dlg.setLabelText(tr("Isleniyor: %1").arg(ogrenciler[i].ad()));
    karneUret(ogrenciler[i]);
    QCoreApplication::processEvents();
}
dlg.setValue(ogrenciler.size());  // tamamlandi
```

Alıştırma

1. `minimumDuration` property'si ne işe yarar? Bu özellik kullanıcı deneyimini nasıl iyileştirir?

Cevabın: _____

2. Bir döngünün içinde kullanıcının iptal butonuna basıp basmadığını nasıl kontrol edersin? Hangi metot kullanılır ve döngü içinde processEvents neden çağrılmalı?

Cevabın: _____

Cevap Anahtarı

1. `minimumDuration` (varsayılan 4000 ms), diyalogun açılması için en az kaç ms beklenmesi gerektiğini belirtir. İşlem bu sürenin altında biterse diyalog hiç görünmez. Bu sayede yarım saniyede biten bir görev için bile ekrana "flaş" gibi bir pencere açılmaz; sadece gerçekten uzun süren işlerde kullanıcı ilerleme görür. Daha agresif hissedilen bir UX için 200-500 ms'ye düşürülebilir.
2. `if (dlg.wasCanceled()) break;` ile her iterasyonda kontrol edilir. Tek thread'li UI'da işlem döngüsü event loop'u bloklar; iptal butonunun tıklamasının işlenmesi için `QCoreApplication::processEvents()` çağrısı şarttır. Bu çağrı bekleyen olayları (tıklama, çizim) işleyip `wasCanceled()` bayrağını günceller. Daha temiz çözüm: işi ayrı bir QThread veya `QtConcurrent::run` ile çalıştırmaktır (*(bkz. kitap)*).

8.8 QErrorMessage

QObject → QWidget → QDialog → QErrorMessage

Tekrarlanan hata mesajlarını biriktiren özel diyalog. Klasik QMessageBox::critical her çağrıda yeni bir pencere açar; OBYS'de aynı doğrulama hatası yüz kez tetiklendiğinde ekran felakete döner. QErrorMessage bunun yerine tek bir kalıcı pencere kullanır, "Bu mesajı bir daha gösterme" onay kutusu sunar ve kullanıcı buna tıklarsa aynı metin tekrar gösterilmez. Ayrıca qtHandler() ile uygulamanın tüm `qWarning()/qCritical()` mesajlarını otomatik yakalayabilir.

Static Metotlar

- `QErrorMessage* QErrorMessage::qtHandler()` – Qt mesajlarını yakalayan global örnek

Slots

- `showMessage(const QString& message)`
- `showMessage(const QString& message, const QString& type)`

Davranış

- Diyalog tek nesnedir; arka arkaya gelen mesajlar kuyruğa alınır
- Kullanıcı "Tamam" deyince bir sonraki mesaj gösterilir
- "Bu mesajı bir daha gösterme" işaretliyse ileride yok sayılır
- `type` verildiğinde aynı tipteki mesajlar tek anahtar altında gruplanır

C++ - Mini ornek: OBYS global hata yakalayicisi

```cpp
// main.cpp
int main(int argc, char* argv[]) {
    QApplication app(argc, argv);
    // Tum qWarning/qCritical mesajlarini diyaloga yonlendir
    QErrorMessage::qtHandler();

    MainWindow w;
    w.show();
    return app.exec();
}

// Baska bir yerde:
qWarning() << "Ogrenci numarasi gecersiz:" << numara;
// Otomatik olarak QErrorMessage diyalogunda gozukur.
```

Alıştırma

1. QErrorMessage ile QMessageBox::critical arasındaki temel kullanım farkı nedir? Hangi senaryoda hangisi tercih edilmeli?

Cevabın: _____

2. "Bu mesajı bir daha gösterme" onay kutusu nasıl çalışır? Aynı oturum içinde mi yoksa kalıcı olarak mı geçerlidir?

Cevabın: _____

Cevap Anahtarı

1. QMessageBox::critical *tek seferlik*, kullanıcının dikkat etmesi şart kritik durumlar için uygundur ("Veritabanına bağlanılamadı, çıkılacak"). Her çağrı yeni bir modal pencere açar; arka arkaya çağrılırsa kullanıcı pencere çığına maruz kalır. QErrorMessage ise sık tekrar edebilen, çoğu zaman göz ardı edilebilir uyarılar için tasarlanmıştır ("Geçersiz not formatı: 105"). Mesajları kuyruğa alır ve kullanıcıya susturma seçeneği sunar.
2. "Bu mesajı bir daha gösterme", QErrorMessage nesnesinin kendi belleğinde tutulan bir set'e o mesaj metnini (veya `type` anahtarını) ekler. Aynı QErrorMessage örneği ileride aynı metni showMessage ile gösterirse sessizce yutulur. Bu *oturum içi* bir filtredir; uygulama kapanıp tekrar açıldığında bayrak sıfırlanır. Kalıcı saklama istersen QSettings'e kendin yazmalısın.

Kategori Bitiş Quizi

5 Soruluk Karma Quiz

1. (Boşluk doldurma) Bir QDialog'un modal şekilde çalışıp kullanıcı kapatana kadar çağıran kodu bloklaması için _____ metodu kullanılır.

2. (Eşleştirme) Aşağıdaki senaryoları en uygun diyalog sınıfıyla eşleştir:

`.obys` dosyası açmak	?
Kayıt öncesi onay almak	?
Karanlık mod için vurgu rengi seçmek	?
Yeni ders adı (kısa metin) almak	?

3. (Kod okuma) Aşağıdaki çağrıda kullanıcı diyalog açıldıktan sonra *Cancel*'a basarsa `yol` değişkeninin değeri ne olur?

C++

```
QString yol = QFileDialog::getOpenFileName(this,
    "Ac", QString(), "OBYS (*.obys)");
```

_____

4. QMessageBox::question çağrısında verilen `defaultButton` parametresi diyalogun davranışında neyi etkiler?

_____

5. (Mikro kod) 0–100 aralığında ilerleyen, başlangıç değeri 0 olan bir QProgressDialog'u oluşturup gösteren 2 satır kod yaz:

_____

Cevap Anahtarı

1. exec(). Bu metot kendi yerel event loop'unu başlatır ve accept() ya da reject() çağrılana kadar çağıran koda dönmez.
2. Eşleştirmeler: `.obys` aç → QFileDialog; kayıt öncesi onay → QMessageBox (question); karanlık mod rengi → QColorDialog; yeni ders adı → QInputDialog (getText).
3. `yol` *boş* bir QString olur (`yol.isEmpty() == true`). Bu yüzden statik dosya diyaloglarından sonra her zaman `if (!yol.isEmpty())` kontrolü yapılmalıdır.
4. `defaultButton`, kullanıcı diyalog açılır açılmaz *Enter* tuşuna bastığında hangi butonun tıklanmış sayılacağını belirler. Aynı zamanda klavye odağı bu butona verilir ve buton görsel olarak vurgulanır. Tehlikeli işlemlerde (ör. "Sil") varsayılanı `Cancel` yapmak iyi bir UX prensibidir.
5. İki satırlık çözüm:

C++

```
QProgressDialog dlg(tr("Isleniyor..."), tr("Iptal"), 0, 100, this);
dlg.setValue(0); dlg.show();
```